

High-Level Design (HLD) / System Design Interview Guide

Seven foundational topic areas, a universal delivery framework, a cross-cutting scenario/trade-off question bank, and five complete worked case studies — URL Shortener, Chat System (WhatsApp), News Feed (Twitter), Distributed Rate Limiter, and Notification System — each carried from requirements through architecture and failure-mode analysis. Categories, question selection, and case studies were dictated entirely by current interview-report and practitioner sources. Targeted at **Mid-level**, with Senior/Staff deep-dives marked throughout.

7 foundational topic areas + 5 full case studies

Mermaid architecture & sequence diagrams

Capacity estimation, rubric, cheat sheet included

Sourced from Hello Interview, GfG, Glassdoor, system-design-primer, AWS docs

Updated Sep 2026

HOW HLD DIFFERS BY LEVEL & TIME-BOXING

Intern/New Grad

Mid

Senior

Staff

- **Mid-level** — produce a coherent architecture that satisfies the stated requirements, with basic scaling (replicas, cache, a sensible data store) reasoned correctly.
- **Senior** — proactively surface failure modes, trade-offs, and operational concerns (monitoring, rollout) without being asked; push the design under 10x/100x load unprompted.
- **Staff/Principal** — reasons about the system's interaction with teams, migrations, cost, and long-term strategy, not just the diagram; a clean architecture with no operational or organizational reasoning is actively downleveled at this bar.

45–60 min: 5 min requirements → 5 min capacity estimate → 15 min high-level architecture → 15–20

WHAT INTERVIEWERS EVALUATE

- **Problem framing** — clarifying users, use cases, constraints before designing anything.
- **Architecture coherence** — APIs, data model, and components that actually fit together, not just a list of buzzwords.
- **Depth and trade-offs** — explaining *why* one option fits better than another for these specific requirements.
- **Production readiness** — capacity, security, reliability, observability, and cost, not just "happy path" architecture.

min deep dive on 1–2 components (interviewer-steered) → 5–10 min failure modes/trade-offs. **90 min variant** adds a second deep-dive component and an explicit "now design for 100x" follow-up round.

00 How HLD Interviews Work

Before any theory: what actually happens in the room, what's being scored underneath the diagram, and the two skills — leading through ambiguity, defending a trade-off without digging in — that current sources say separate a passing interview from a failing one, independent of the architecture itself.

Common interview formats

FORMAT	WHAT IT TYPICALLY MEANS
45 minutes	One system, one pass: requirements → architecture → one deep dive. Little room for a second component.
60 minutes	The default. Room for a real deep dive on 1–2 components plus a scaling/failure-mode push.
90 minutes	Either a second full deep-dive component, or the same system pushed much further (multi-region, 100x load) — testing depth, not breadth.

How to lead the conversation and avoid getting stuck

Sources are consistent and specific here: **the interviewer expects you to lead**. "When you're asked a vague problem, it's your job to clarify scope, define requirements, and drive the conversation forward" — waiting for the interviewer to prompt each next step reads as passivity, not caution. The recommended shape: start with a simple, high-level architecture before drilling into any one component ("give the high-level design first, then drill down") — going deep on one box before the overall shape exists is a documented way to run out of time with an incomplete picture.

If you genuinely get stuck, the guidance is blunt: **say so, out loud**. "They can't tell that you're at an impasse unless you tell them... don't think in silence." Naming the specific fork you're stuck on ("I'm deciding between a queue and a direct call here, and I'm not sure which the requirements favor — let me think through it") is treated as a positive, recoverable moment, not a failure — silence is the actual failure mode.

Handling ambiguous and changing requirements

System design prompts are deliberately open-ended, "to assess your ability to design complex systems, handle uncertainty, and communicate effectively." The standard opening move: spend the first 5–10 minutes of an hour-long interview on clarifying questions — enough to fix scope without eating into design time. Two techniques sources specifically call out as strong signals: **restating requirements in your own words** ("so to confirm: we need X, Y, and Z, and Q is explicitly out of scope") — this shows active listening and gives the interviewer a chance to correct a misunderstanding before you've designed around it; and **making an assumption explicit rather than silently guessing** when information genuinely isn't available.

When a constraint changes mid-interview (a classic move: "actually, load just doubled" or "we now need this to work offline"), the weak response is either panic or a full restart. The strong response, per current sources, is **diagnostic**: ask what specifically changed ("did user count increase, or did request size grow? is the cache hit rate dropping?") before deciding what in the existing design needs to adapt — treating the new information as a targeted update to a still-mostly-valid design, not a signal to discard everything and start over.

Defending a trade-off under pushback

This is named directly as a specific, distinct failure mode: "when the interviewer pushes back on your database choice, instead of reasoning through it, candidates often dig in and defend the original choice... candidates who fail usually fall into a sunk-cost mindset, defending outdated assumptions simply because they already drew the boxes." The fix isn't having a better initial answer — every real design decision has genuine downsides, and claiming otherwise reads as naive, not confident. The fix is a specific verbal pattern current sources hold up as the model response:

"I am choosing eventual consistency here to get availability and scale, at the cost of occasionally serving slightly stale data, which is acceptable for a feed."

Naming the trade-off explicitly, before the interviewer even has to push, defuses the pushback into a collaborative conversation rather than an adversarial one — "interviewers like people who can think collaboratively... under pressure, most candidates treat pushback as an attack" when it's actually intended as a probe of exactly this skill.

WHAT THE WEB SOURCES EMPHASIZE

Independent sources converge on the same root diagnosis for interview failure: "candidates rarely fail because they cannot design systems — they fail because they misunderstand the evaluation criteria," specifically underestimating how much of the score is the live conversation (leading, adapting, defending) rather than the final diagram's correctness.

01 Scalability & Networking Fundamentals

Sources agree this is the entry gate: load balancing, horizontal vs. vertical scaling, and where a request actually flows before it reaches your service — CDN/edge, then load balancer/API gateway.

CORE DEFINITIONS

Vertical scaling ("scale up")

Adding more resources (CPU, RAM) to one existing machine — simple, but hits a hard ceiling and creates a single point of failure.

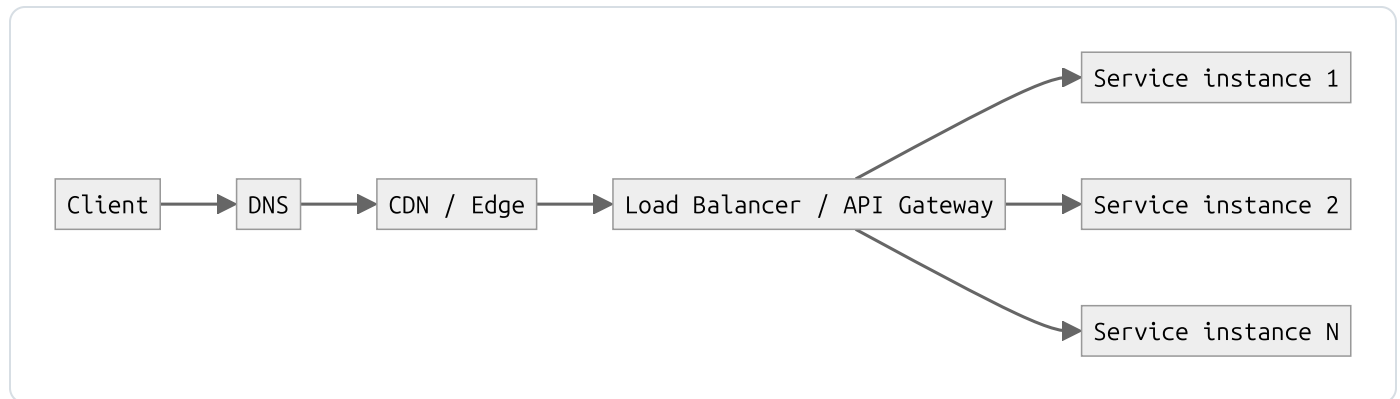
Horizontal scaling ("scale out")

Adding more machines and distributing load across them — the default assumption in almost every modern HLD answer, since it has no inherent ceiling and improves fault tolerance.

Load balancer

A component that distributes incoming requests across multiple backend servers so no single server is overwhelmed — described in sources as "a traffic cop sitting in front of your servers."

1.1 BASIC What happens to a request before it reaches your service, end to end?



Per current sources, the standard modern flow is: DNS resolves to the nearest edge, the request hits a **CDN/edge** layer first (serving cached static content, terminating TLS, applying basic DDoS/rate-limiting), then an unmet request is forwarded to a **load balancer or API gateway**, which routes it to one of many stateless service instances. Naming this full chain unprompted — not just "there's a load balancer" — is a cheap, strong opening signal.

CDN/edge

load balancer

API gateway

1.2 BASIC Vertical vs. horizontal scaling — when do you actually choose vertical?

	VERTICAL SCALING	HORIZONTAL SCALING
Mechanism	Bigger machine (more CPU/RAM)	More machines, load-balanced
Ceiling	Hard hardware limit	No inherent ceiling
Fault tolerance	Single point of failure	One instance's failure doesn't take the system down
Complexity	None — no distributed-systems concerns	Requires statelessness, data partitioning, coordination

Vertical scaling is still the right first move for a genuinely stateful, hard-to-partition component (a single primary relational database, early on) or when the team's actual bottleneck is engineering time, not request volume — it buys headroom with zero added distributed-systems complexity. The moment a single machine's ceiling is a real, foreseeable constraint (not a hypothetical), horizontal scaling becomes necessary — this is a trade-off to state explicitly, not an automatic "always scale horizontally" reflex.

scale up vs scale out

single point of failure

1.3 INTERMEDIATE Load balancing algorithms and where a load balancer sits (L4 vs L7).

ALGORITHM	MECHANISM	BEST FOR
Round robin	Cycles through servers in order	Roughly equal-cost requests, uniform server capacity
Least connections	Routes to the server with fewest active connections	Highly variable request duration/cost
Consistent hashing	Hashes a request key to consistently route to the same backend	Cache affinity, session stickiness, minimal reshuffling on scale events

An **L4 (transport-layer)** load balancer routes on IP/port alone — fast, protocol-agnostic, but blind to request content. An **L7 (application-layer)** load balancer reads the actual HTTP request (path, headers, cookies) to make smarter routing decisions — URL-based routing, canary releases by header — at the cost of more overhead per request. Most modern HLD answers default to L7 for anything HTTP-based, reserving L4 for raw TCP/UDP workloads.

round robin least connections consistent hashing L4 vs L7

1.4 ADVANCED

Why must services behind a load balancer be stateless, and what do you do with state that has to exist?

Horizontal scaling only works cleanly if any instance can handle any request — the moment a service instance holds request-specific state in local memory (a user's session, an in-progress upload), the load balancer either has to pin that user to that one instance (**sticky sessions** — reintroducing a single point of failure for that user) or requests randomly fail depending on which instance they land on. The standard fix: push state out of the service instance entirely, into a shared, external store — a distributed cache (Redis) for session data, a database for durable state, object storage for large blobs — so every service instance is interchangeable and stateless. This is one of the most consistently tested "why" questions in HLD interviews, precisely because it's the structural assumption every other scaling technique in this guide depends on.

stateless services sticky sessions externalized state

WHAT THE WEB SOURCES EMPHASIZE

GeeksforGeeks and multiple 2026 prep guides converge on the same request path — CDN/edge first, then load balancer/API gateway, then stateless services — as the expected default mental model candidates should reach for immediately, before any case-study-specific reasoning begins.

COMMON MISCONCEPTION

Treating "add a load balancer" as a complete answer on its own. The load balancer only works because the services behind it are stateless — skipping that dependency is the single most common gap in an otherwise correct-looking diagram.

02 API Design (REST, GraphQL, gRPC)

Sources call API gateway questions "the most common in system design loops because a gateway is where most cross-cutting concerns live." This section covers both the external contract and the internal service-to-service protocol choice.

2.1 BASIC What does an API gateway actually do, beyond routing?

Per current sources, a gateway is a single entry point in front of many backend services, and its real value is centralizing cross-cutting concerns so individual services don't each reimplement them: **routing** (mapping a path/host to a backend), **authentication** (checking the caller once, not per-service), **rate limiting** (capping call frequency per client, Section 13's case study), **aggregation** (calling several services and merging results for the client in one round trip), and **protocol translation** (accepting REST from a browser, speaking gRPC internally). Naming these five responsibilities specifically, rather than "it routes requests," is what separates a memorized diagram from real understanding.

API gateway

cross-cutting concerns

protocol translation

2.2 BASIC REST vs. GraphQL vs. gRPC — when to use which?

	REST	GraphQL	gRPC
Data shape	Fixed per endpoint — over/under-fetching common	Client specifies exactly the fields needed	Fixed, strongly-typed (Protocol Buffers)
Performance	Text (JSON), moderate overhead	Text (JSON), can be chattier server-side	Binary, low overhead, HTTP/2 multiplexed
Best for	Public/third-party APIs — universal, cacheable, well-understood	Mobile/web clients with varied, nested data needs (avoid many round trips)	Internal service-to-service calls — latency and strict contracts matter more than human-readability

The commonly cited real-world composite architecture: **REST or GraphQL at the edge** for browser/mobile clients (browsers can't speak gRPC directly), **gRPC internally** between microservices in a high-volume, possibly multi-language environment, with the gateway bridging the two — this is the answer sources point to as the practical default, not a single protocol "winning" universally.

REST GraphQL gRPC Protocol Buffers

2.3 INTERMEDIATE

Design the external API for a URL shortener (a concrete, gradable example).

```
POST /api/v1/urls
  body: { "longUrl": "https://example.com/very/long/path", "customAlias": null, "ttl": null }
  201 → { "shortUrl": "https://short.ly/aZ9kQ2", "expiresAt": null }
  400 → invalid URL   429 → rate limited

GET /{shortCode}
  302 → Location: (redirect, not a JSON response)
  404 → short code not found or expired

GET /api/v1/urls/{shortCode}/stats
  200 → { "clicks": 4821, "createdAt": "...", "lastAccessedAt": "..." }
```

Two design decisions worth narrating explicitly: the redirect endpoint returns a real HTTP **302**, not a JSON payload — this is the whole point of a URL shortener, letting browsers follow it natively with zero client-side logic; and creation is a **POST**, not idempotent by default, which sets up a direct follow-up on what happens if a client retries a timed-out creation request (Section 9's idempotency-key pattern, reused directly from the LLD guide's reliability section).

302 redirect API versioning (/v1/)

2.4 ADVANCED

How do you design an API that must evolve without breaking existing clients?

Two complementary tools, both expected at senior level: **versioning** (a `/v1/`, `/v2/` path prefix, or a version header) lets you introduce breaking changes as an explicitly new contract while old clients keep hitting the old one, with a deliberate, communicated deprecation window rather than a surprise cutover. **Backward-compatible additive changes** — adding new optional fields to a response, never removing or repurposing existing ones — let most evolution happen without bumping the version at all, since well-behaved clients ignore unknown fields. The senior-level nuance worth stating: versioning an entire API is a heavy, coordination-costly tool; most real API evolution should be additive, with hard versioning reserved for genuine breaking changes that can't be made additive.

API versioning

backward compatibility

deprecation window

WHAT THE WEB SOURCES EMPHASIZE

Hello Interview's API design material and multiple 2026 guides converge on the same composite architecture (REST/GraphQL at the edge, gRPC internally, gateway as the bridge) as the expected default — this shaped 2.2's structure directly rather than presenting the three protocols as equally likely choices for any given layer.

03 Data Storage: Sharding, Replication & Consistency

Sources are explicit that CAP is where "surface understanding" gets separated from "a real one" — the test is whether a candidate understands partition tolerance is non-negotiable, not a free third option.

CORE DEFINITIONS

Replication

Keeping copies of the same full data set on multiple machines — for redundancy (a failed machine doesn't lose data) and read scaling (multiple machines serve read traffic).

Sharding (partitioning)

Splitting the data set into disjoint subsets, each server owning one or more shards — for write scaling and dataset-size scaling beyond one machine.

3.1 BASIC State the CAP theorem, precisely.

A distributed data store can guarantee at most two of three properties simultaneously: **Consistency** (every node sees the same data at the same time — a read always returns the latest write), **Availability** (every request receives a non-error response), and **Partition tolerance** (the system keeps operating despite network messages being dropped or delayed between nodes). The precise, interview-grade framing sources emphasize: partition tolerance isn't a design choice you opt out of — in any real multi-node system, network partitions *will* happen — so the actual decision only matters *during* an active partition, and it's really a choice between Consistency and Availability at that moment, not a free pick of any two. "Choosing CA" only describes a single-node system, which by definition never experiences an internal network partition.

CAP theorem partition tolerance is mandatory

WHAT INTERVIEWERS LISTEN FOR

Sources state this directly: "recognizing partition tolerance is mandatory and the real choice is C-vs-A during a partition — not during normal operation — is what separates a surface understanding from a real one."

3.2 BASIC Strong vs. eventual consistency — with a concrete example of each failing.

	STRONG CONSISTENCY	EVENTUAL CONSISTENCY
Guarantee	A read always returns the latest write, from any node	A write is accepted quickly; all nodes converge over time
Cost	Higher latency (must coordinate across nodes/replicas)	Lower latency; reads may return stale data temporarily
Fails visibly as	Slow writes/reads, reduced availability during a partition	A user seeing their own just-made change disappear on refresh
Good fit	Bank balance, inventory count, a "like" that gates a payment	A social media like count, a view counter, a news feed

The concrete question interviewers actually want answered isn't "define both" — it's "which one does *this specific field in this specific system* need," decided per data type, not once for the whole system. A single system commonly needs both: strong consistency for account balances, eventual consistency for a follower count.

strong consistency eventual consistency per-field consistency choice

3.3 INTERMEDIATE SQL vs. NoSQL — the real decision criteria, not a popularity contest.

	SQL (RELATIONAL)	NOSQL (DOCUMENT/KV/WIDE-COLUMN)
Schema	Fixed, enforced	Flexible/schemaless
Transactions	ACID, multi-row/table by default	Often limited to single-document/single-partition atomicity
Relationships	Native JOINS	Denormalized/embedded, or resolved in application code
Scaling pattern	Vertical first, sharded with real effort	Designed for horizontal scale from the start
Best fit	Complex relationships, multi-record transactions (orders, payments)	Very high write throughput, flexible/evolving shape, massive horizontal scale (chat messages, event logs, session data)

The senior-level answer avoids "NoSQL scales better" as a slogan and instead names the actual trade-off: relational databases enforce correctness guarantees that NoSQL systems push onto the application in exchange for easier horizontal scaling. Real production systems are commonly **polyglot** — a relational store for the transactional core (orders, billing) and a NoSQL store for a specific high-volume, simpler-shape workload (chat history, activity feed) side by side.

polyglot persistence

ACID vs. BASE

3.4 ADVANCED

How do you choose a shard key, and what's the most common way to get it wrong?

Per current sources, "the most common mistake is choosing a shard key that works for one access pattern while destroying another." A good shard key distributes both storage and load evenly across shards *and* lets the majority of real queries be satisfied by a single shard. The canonical trap: sharding a Users table by `signup_date` looks reasonable, but concentrates all of *today's* write and read traffic onto one "hot" shard while older shards sit idle — the opposite of the even distribution sharding is supposed to provide. Sharding by a well-distributed key like `user_id` (via consistent hashing, 1.3) avoids this, at the cost of making "all users who signed up today" a cross-shard query. The rule to state explicitly: the shard key should be chosen from the dimension real query traffic is actually distributed across, not whatever column happens to be convenient — and every design should name which query patterns become cross-shard (and therefore expensive) as a result.

shard key

hot shard

cross-shard query

COMMON MISCONCEPTION

Assuming a shard key that's good for writes is automatically good for reads too — the two access patterns are frequently in tension, and a strong answer names the specific query type sacrificed by the chosen key.

WHAT THE WEB SOURCES EMPHASIZE

Multiple current sources (Hello Interview's CAP theorem page, Data Interview's distributed-systems question bank) independently flag the exact same two failure points: candidates reciting CAP as "pick any two" instead of understanding partition tolerance as mandatory, and choosing a shard key that optimizes one access pattern while silently breaking another.

04 Caching Strategies

Sources frame this precisely: "interviewers evaluate whether you can design caching that improves performance without introducing stale data bugs" — the strategy choice is the whole exam, not a side detail.

4.1 BASIC Cache-aside vs. read-through vs. write-through vs. write-back.

STRATEGY	READ PATH	WRITE PATH	TRADE-OFF
Cache-aside	App checks cache; on miss, reads DB and populates cache itself	App writes DB directly; cache is invalidated/left to expire	Simple, most common; a miss storm can hit the DB hard
Read-through	Cache itself loads from DB on a miss, transparently	Same as cache-aside	Simpler application code; cache library must support it
Write-through	—	Written to cache and DB simultaneously	Cache and DB always consistent; every write pays cache-write latency
Write-back (write-behind)	—	Written to cache immediately; DB write deferred/batched	Fastest writes; real data-loss risk if the cache fails before flushing

Cache-aside is the default most systems reach for first — simple, and the application stays in full control of exactly when the cache is populated. Write-through is chosen specifically when the cache and database must never disagree even briefly; write-back is chosen only when write latency is the dominant constraint and the team has explicitly accepted the data-loss window as a trade-off.

cache-aside

write-through

write-back

4.2 INTERMEDIATE

Cache invalidation — why is it "one of the two hard problems in computer science," and what actually works?

- **TTL (time-to-live)** — simplest: every cached entry expires after a fixed duration, bounding staleness without any explicit invalidation logic. The trade-off is a direct dial between freshness and cache-hit rate/DB load (Section 4.3, LLD's Section 4.3 on DNS TTL makes the identical trade-off).
- **Event-driven invalidation** — the write path explicitly deletes or updates the affected cache entry the moment the underlying data changes; no staleness window at all, at the cost of needing every writer to remember to do this correctly.
- **Versioning** — cache keys embed a version number that increments on write, so old cached entries are simply never looked up again rather than needing active deletion.

Real production systems layer these: TTL as a safety net that bounds worst-case staleness even if event-driven invalidation is missed somewhere, plus event-driven invalidation for the common, latency-sensitive path. Naming this layered approach — rather than picking exactly one — is the answer that reads as production experience rather than textbook recall.

TTL

event-driven invalidation

cache versioning

4.3 ADVANCED

What is the thundering herd / cache stampede problem, and how do you prevent it?

When a hot cache entry expires (or the cache itself restarts cold), every concurrent request for that key misses simultaneously and hits the database at once — a spike that can look, to the database, exactly like the traffic surge that necessitated caching in the first place, potentially taking it down. Mitigations worth naming: **request coalescing** (the first miss triggers the DB read; concurrent requests for the same key wait on that in-flight result instead of each issuing their own DB query); **staggered/jittered TTLs** (avoid every related key expiring at the exact same instant); and **stale-while-revalidate** (serve the slightly-stale cached value immediately while asynchronously refreshing it in the background) — the same pattern named directly in current CDN documentation for exactly this purpose.

thundering herd

cache stampede

stale-while-revalidate

WHAT THE WEB SOURCES EMPHASIZE

Multiple 2026 caching guides converge on the same four-strategy taxonomy (cache-aside/read-through/write-through/write-back) as the expected vocabulary, and independently name stale-while-revalidate as the standard modern CDN/edge-caching pattern — used directly in 4.3 rather than a generic "add a cache" answer.

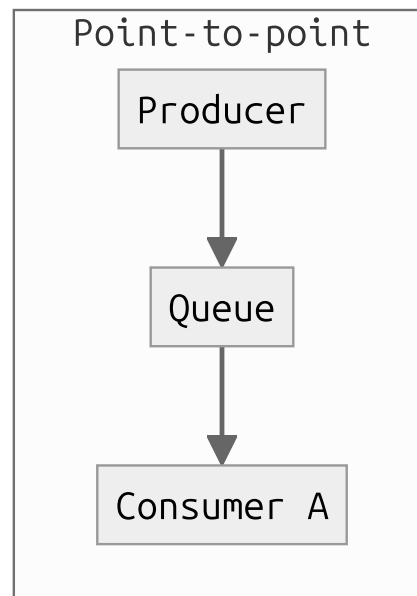
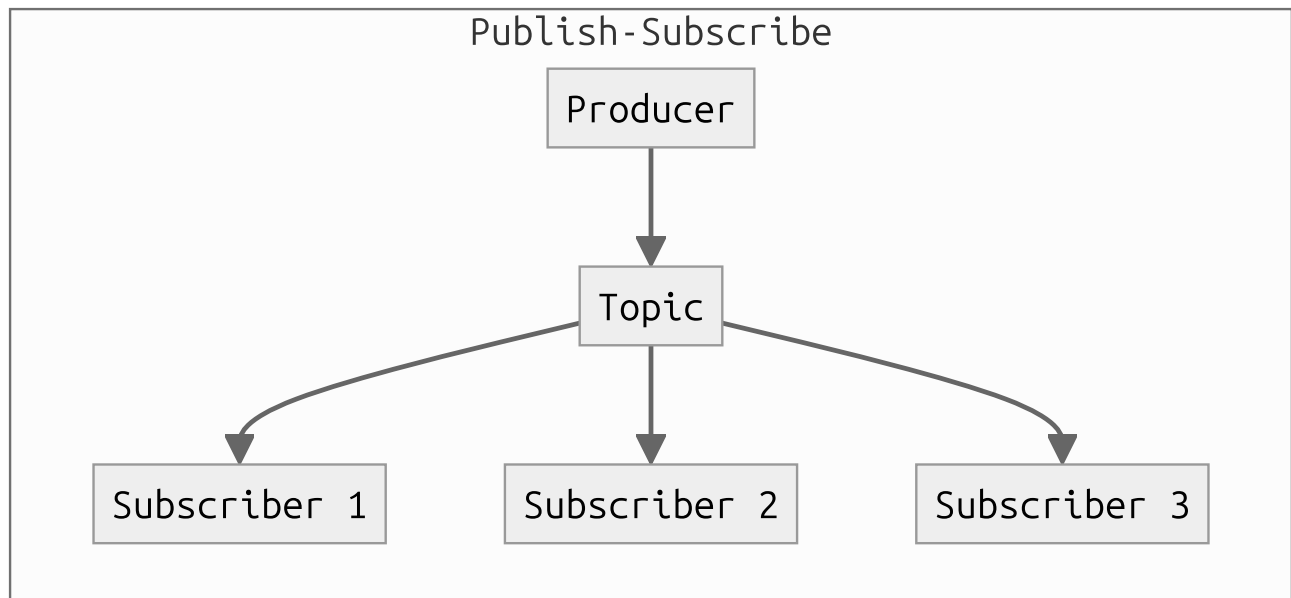
COMMON MISCONCEPTION

Treating "add a cache" as sufficient without naming an invalidation strategy — a cache with no invalidation plan is a design that will demonstrably serve stale data, and interviewers routinely probe exactly this gap.

05 Asynchronous Processing: Queues & Streaming

Every chat/notification/feed case study in this guide leans on this section — queues are the mechanism that decouples "accept the request fast" from "do the actual work."

5.1 BASIC Point-to-point queue vs. publish-subscribe — what's the actual difference?



In a **point-to-point (task) queue**, a message is consumed by exactly *one* of possibly many competing consumers — the right model for distributing work items (image resizing jobs) across a worker pool, where each job should only be done once. In **publish-subscribe**, a message published to a topic is delivered to *every* independent subscriber — the right model when several unrelated systems all need to react to the same event (an order-placed event triggering inventory, billing, and notification services independently, each unaware of the others).

point-to-point

pub-sub

competing consumers

5.2 INTERMEDIATE

Kafka vs. a traditional message queue (RabbitMQ/SQS) — when does the distinction actually matter?

	TRADITIONAL QUEUE (RABBITMQ/SQS)	KAFKA
Model	Message removed once consumed/acked	Durable, ordered log — messages persist and can be re-read
Best for	Task distribution, point-to-point work queues	High-throughput event streaming, multiple independent consumer groups replaying the same stream
Replay	Not natively supported	Native — a new consumer can start from any offset, including the beginning
Ordering	Per-queue, generally	Per-partition, strictly ordered

The distinction matters concretely whenever a system needs more than one independent consumer group processing the *same* events at different times or speeds (a real-time fraud-check consumer and a nightly batch-analytics consumer both reading every transaction event) — Kafka's log-based durability supports this natively; a traditional queue that deletes messages on ack does not.

Kafka

durable log

partition ordering

consumer groups

5.3 ADVANCED

How do you handle a poison message, and how do consumers handle duplicates?

A **poison message** is one that a consumer can never successfully process (malformed payload, a bug triggered only by this specific input) — without a circuit breaker on retries, it can be redelivered and re-fail forever, blocking the queue behind it. The standard pattern: retry with exponential backoff for a bounded number of attempts, then route the message to a **Dead-Letter Queue (DLQ)** for manual inspection or a separate remediation process, rather than blocking the main queue indefinitely.

Duplicate delivery is a structural property of most real queue systems under "at-least-once" delivery semantics (a consumer crashes after processing but before acking, so the message is redelivered) — consumers must therefore be built to handle duplicates via **idempotency**: a unique message ID checked against a dedup store before processing, so reprocessing the same message twice has no additional effect. This is the exact same idempotency-key pattern used for API retries (2.3) and in the LLD guide's reliability section — the same structural problem recurring at the messaging layer.

dead-letter queue

exponential backoff

at-least-once delivery

idempotent consumer

WHAT THE WEB SOURCES EMPHASIZE

Hello Interview's Kafka deep-dive and multiple queue-focused guides converge on the same core distinction — durable/replayable log vs. delete-on-ack queue — as the actual decision driver, and independently name DLQs plus consumer-side idempotency as the two production concerns interviewers expect unprompted at anything above entry level.

06 Reliability, Observability & SRE

Explicitly named by sources as a level-defining section: a design with a clean architecture but no operational reasoning gets **downleveled** at the Senior/Staff bar, regardless of how correct the diagram is.

CORE DEFINITIONS

SLI (Service Level Indicator)

The actual measured number for a system's health — e.g. "99.95% of requests succeeded in the last 30 days."

SLO (Service Level Objective)

The internal target for an SLI — what "healthy" is defined to mean, e.g. "99.9% success over a rolling 30-day window."

SLA (Service Level Agreement)

An external, contractual commitment (usually looser than the internal SLO) with real business consequences — credits, penalties — if missed.

6.1 BASIC What are the four golden signals?

SIGNAL	MEASURES	EXAMPLE
Latency	How long requests take	p50/p99 response time — p99 under 300ms
Traffic	Demand on the service	Requests per second
Errors	Rate of failed requests	5xx rate, timeout rate
Saturation	How full the system is	CPU/memory utilization, queue depth

Per the Google SRE book (via current sources), these four are the minimum viable dashboard for any service — naming all four unprompted, with a concrete example metric for each, is a strong, cheap signal in the observability portion of an interview.

golden signals

p99 latency

6.2 INTERMEDIATE Error budgets — what problem do they actually solve?

An **error budget** is the mathematical inverse of the SLO ($100\% - \text{SLO}$) — a 99.9% SLO leaves a 0.1% error budget. Its real purpose, per current sources, is resolving the perpetual tension between shipping features and maintaining reliability *without a subjective argument each time*: as long as the error budget isn't exhausted, the team is free to ship faster and take more risk; once it's burned down, feature work pauses in favor of reliability work, by policy, not by debate. This reframes "how reliable should we be" from a philosophical question into a number both product and engineering can agree on in advance — naming this framing (not just the arithmetic) is what signals real SRE exposure.

error budget

SLO-driven prioritization

6.3 ADVANCED

Design a two-window alerting strategy for an SLO
— why not just alert the instant the SLO is at risk?

Alerting the instant any single request fails is far too noisy (transient blips happen constantly and don't threaten the SLO); alerting only once the full 30-day budget is already exhausted is far too late to act. The standard pattern named directly in current sources: a **fast-burn alert** fires when the error budget is being consumed at roughly 10x its sustainable rate over a short window (e.g. 1 hour) — catching a sudden incident quickly. A **slow-burn alert** fires at a lower burn multiple (e.g. 2x) over a longer window (e.g. 6 hours) — catching a gradual degradation that a short window would miss entirely. Naming both windows, and explaining why one alone is insufficient, is a specific, current, senior-level answer.

fast-burn alert slow-burn alert error budget burn rate

6.4 ADVANCED Design a disaster-recovery plan for a regional outage.

STRATEGY	RTO/RPO	COST
Backup & restore	Hours (RTO), up to last backup (RPO)	Lowest
Pilot light	Tens of minutes	Low — minimal standby infra always running
Warm standby	Minutes	Moderate — scaled-down but running replica region
Multi-region active-active	Seconds / near-zero	Highest — full duplicate capacity, active traffic in both regions

RTO (Recovery Time Objective — how long can we be down) and **RPO** (Recovery Point Objective — how much data can we afford to lose) are the two numbers that should drive this choice, not a reflexive "just go multi-region active-active." A strong answer picks a strategy proportional to the actual business cost of downtime for this specific system, states the resulting RTO/RPO explicitly, and names the real cost trade-off of going further — this is exactly the kind of proportionality reasoning the Staff-level rubric (Section 15) rewards.

RTO RPO multi-region active-active warm standby

WHAT THE WEB SOURCES EMPHASIZE

Current sources are unusually direct here: "a candidate who produces a clean high-level architecture is downleveled, because the interviewer expects proactive discussion of failure modes, operational concerns, and cost reasoning without being asked" — this is why every case study in Sections 10–14 has a dedicated Reliability & DR subsection rather than treating it as optional.

07 Security & Threat Modeling

Sources frame security-focused rounds around three questions: does the design prevent the most likely abuse paths, contain the blast radius when something goes wrong, and leave an investigation trail?

7.1 BASIC Authentication vs. authorization — and the standard protocols for each.

Authentication answers "who are you" — verifying identity, typically via **OAuth 2.0/OIDC** (delegated third-party login) or **SAML** (common in enterprise SSO). **Authorization** answers "what are you allowed to do" — enforced via access-control models: **RBAC** (role-based — permissions attached to roles like "admin," "editor"), **ABAC** (attribute-based — rules over arbitrary attributes, e.g. "only during business hours, from a corporate IP"), or a centralized **policy engine** for complex, evolving rules. The principle sources name explicitly: **least privilege** — every new piece of functionality gets exactly the access it needs, never more, by default.

OAuth 2.0 OIDC RBAC ABAC least privilege

7.2 INTERMEDIATE

What does zero-trust architecture mean for a microservices system specifically?

The traditional model trusts anything already inside the network perimeter; **zero-trust** rejects that assumption entirely — every service must independently authenticate and authorize *every* call it receives, including calls from other internal services, not just calls from outside the network boundary. Concretely, this means service-to-service calls carry short-lived, verifiable credentials (mutual TLS certificates, or short-lived tokens) rather than relying on "it's on the internal network, so it's trusted." The direct payoff: a single compromised service can't silently call every other internal service with unlimited trust — its blast radius is contained to whatever it was explicitly authorized to reach.

zero-trust mutual TLS blast-radius containment

7.3 ADVANCED Walk through a lightweight threat-modeling pass for a new feature.

1. **Identify key assets** — what's actually valuable/sensitive here (user credentials, payment data, private messages)?
2. **Identify entry points** — every place an attacker's input reaches the system (public API endpoints, file uploads, webhooks from third parties).
3. **Identify top threats per entry point** — a public "create post" endpoint: abuse via spam/bots (mitigated by rate limiting, Section 13), injection if input reaches a query unsanitized, stored XSS if content is later rendered unsafely.
4. **Name mitigations** — rate limiting, short-lived revocable tokens instead of long-lived static keys, a centralized policy engine rather than scattered per-service authorization checks, input validation/sanitization at the boundary.
5. **Ensure an investigation trail exists** — audit logs of who accessed/changed what, retained long enough to actually investigate an incident after the fact.

This five-step pass is deliberately lightweight enough to run in a few minutes inside an interview — the point isn't exhaustive enterprise threat modeling, it's demonstrating the habit of asking "how could this be abused" before declaring a design finished.

threat modeling

entry points

audit trail

WHAT THE WEB SOURCES EMPHASIZE

Current sources converge on the same short checklist — OAuth/OIDC/SAML for authentication, RBAC/ABAC for authorization, zero-trust for service-to-service calls, and OWASP Top 10 awareness (XSS, SQLi, CSRF) — as the expected baseline, with threat modeling framed explicitly around abuse paths, blast radius, and investigation trail rather than an exhaustive enterprise process.

08 Universal HLD Problem-Solving Framework

Synthesized from Hello Interview's own Delivery Framework and the recurring structure across dozens of aggregated interview reports. Every case study in Sections 10–14 follows exactly these steps, in order.

Step 1 — Clarifying questions & assumptions (first 3–5 minutes, always)

Sources are blunt: "system design prompts are intentionally open-ended and unstructured, mirroring real product requirements." Never assume — ask, then explicitly state whatever you

proceed on: primary use cases, scale (users, requests/day), read-heavy or write-heavy, must-haves vs. nice-to-haves, what's explicitly out of scope for this session.

Step 2 — Functional & non-functional requirements (with SLOs)

REQUIREMENTS CHECKLIST TEMPLATE

Functional (what the system does):

- [] Core user actions (list 3-5 max – resist scope creep)
- [] Who are the actors (end users, internal services, admins)?

Non-functional (how well it does it):

- [] Availability target (e.g. 99.9% → ~8.7 hrs downtime/year)
- [] Latency target (e.g. p99 < 200ms for reads)
- [] Consistency requirement (strong / eventual – per data type, Section 3.2)
- [] Durability requirement (can we ever lose data? how much?)
- [] Data retention / compliance (GDPR deletion, financial record retention)
- [] Expected scale (DAU, peak QPS, data volume/growth rate)

Step 3 — Capacity estimation (back-of-the-envelope)

CAPACITY ESTIMATION WORKSHEET

QPS (average) = (DAU x actions_per_user_per_day) / 86,400

QPS (peak) = QPS(average) x peak_multiplier // commonly 2x-5x; Pareto: 80% of traffic in 20% of the day

Storage (total) = daily_writes x avg_record_size x retention_days x replication_factor

Bandwidth = data_per_request x QPS x 8 // bits/sec

Servers needed = QPS(peak) / QPS_per_server // benchmark or assume ~1000 simple req/s per instance

Per current sources: "the interviewer wants to see that you can quickly produce reasonable estimates and connect the numbers to design decisions... being off by 2x is fine." The estimate's *purpose* is deciding order of magnitude — do we need 1 server or 100, a single-node database or a sharded cluster — not precision. Skipping this step entirely, or doing it with no connection to any later design decision, are both documented weak signals.

Step 4 — API design (external + internal)

Define the external contract (Section 2) first — it forces clarity on exactly what the system does before any internal architecture is drawn. Internal RPC signatures between your own services come after the high-level architecture (Step 6), once the service boundaries are known.

Step 5 — Data model & major entities

List the core entities and their key fields/relationships — not a full schema, but enough to reason about access patterns, which directly drives Step 6's storage choice (Section 3) and Step 3's capacity numbers.

Step 6 — High-level architecture decomposition

Decompose into services by responsibility (not by technology), draw the request path (Section 1.1's chain as the default skeleton), and name each component's storage/caching/queueing choice explicitly, with a one-line reason per choice.

Step 7 — Consistency approach & correctness argument

For each piece of state introduced in Step 5, state the consistency requirement (Section 3.2) and how the architecture actually delivers it — this is where "I chose eventual consistency for the view counter because staleness there is invisible to users, but strong consistency for the payment record because it isn't" belongs.

Step 8 — Failure modes & resiliency

For each major component, ask explicitly: what happens when this dies? Name the blast radius, the detection mechanism, and the mitigation (retry/backoff, circuit breaker, failover to a replica) — per Section 6, doing this unprompted is a Senior-level expectation, not an optional extra.

Step 9 — Observability, deployment & security pass

ARCHITECTURE REVIEW CHECKLIST TEMPLATE

Observability:

- ☐ Golden signals dashboards per service (Section 6.1)
- ☐ SLO defined, with a two-window alert (Section 6.3)
- ☐ Distributed tracing across service boundaries (request ID propagation)

Deployment / rollout:

- ☐ Can this ship as a canary / staged rollout, not a big-bang release?
- ☐ Is there a fast, tested rollback path?
- ☐ Do any data migrations need to be backward-compatible during rollout?

Security:

- ☐ AuthN/AuthZ model named (Section 7.1)
- ☐ Rate limiting / abuse prevention at the edge
- ☐ Sensitive data encrypted at rest and in transit
- ☐ Threat-model pass done (Section 7.3)

Cost:

- ☐ Any component whose cost scales faster than revenue/usage?
- ☐ Build vs. buy named explicitly for non-core infrastructure

WHAT INTERVIEWERS LISTEN FOR

Narrating *why* at every step, out loud, as you go — "I'm choosing eventual consistency here specifically because a stale view count for a few seconds is invisible to the user, unlike a stale balance" — scores higher than an identical diagram produced silently.

09 Scenario & Trade-off Question Bank

Cross-cutting questions that don't belong to one category — scenario prompts, explicit trade-offs, and incident-style debugging, exactly the three styles current sources report interviewers reaching for beyond the standard case-study prompt.

Scenario questions

9.1 INTERMEDIATE

"Traffic to your service just spiked 10x overnight. Walk through your response." What's being tested?

This tests whether the candidate has actually internalized horizontal scaling (Section 1) as more than a slogan. A strong answer separates **immediate mitigation** (auto-scaling policies already in place should absorb it if the architecture is genuinely stateless; rate limiting/load shedding at the edge protects the system if scaling can't keep pace) from **root-cause investigation** (is this legitimate growth, a retry storm from a struggling downstream dependency, or an abuse pattern?) — conflating the two, or jumping straight to "add more servers" without asking which of the three it is, is the common weak answer.

auto-scaling

load shedding

retry storm

9.2 ADVANCED

"Your primary region just went dark. Walk through what happens, minute by minute."

Tests Section 6.4's DR framework applied under pressure. A strong, minute-by-minute answer: health checks detect the region is unreachable within seconds; DNS/global load balancer fails over traffic to the standby region (naming *how* — Route 53 health-check-based failover, a global anycast load balancer); the standby region's data currency depends on the chosen replication strategy (synchronous replication = zero data loss but higher write latency day-to-day; asynchronous = some recent writes lost, bounded by replication lag, per the DBMS guide's identical trade-off); on-call is paged, and a post-incident review follows. The specific numbers (RTO/RPO) should already have been decided *before* the outage, not improvised during it — naming that this decision predates the incident is itself the strong signal.

regional failover

DNS-based failover

replication lag data loss

9.3 ADVANCED

"One specific key/user is getting 1000x normal traffic (a hot key). Diagnose and fix."

A hot key/hot partition breaks the even-distribution assumption every sharding and caching strategy depends on (Section 3.4) — one shard or one cache node absorbs disproportionate load while the rest of the fleet sits idle. Mitigations, roughly in order of effort: **local/in-process caching** of the hot key on each service instance (accepting some staleness) to avoid hitting the shared cache/DB at all for that key; **read replicas specifically for the hot shard**; **splitting the hot key** into several sub-keys with a suffix (e.g. `celebrity_id:0` through `celebrity_id:9`), writing/reading across all of them and aggregating — directly the same structural idea as the News Feed case study's celebrity-fanout problem (Section 12).

hot key

hot partition

key splitting

9.4 ADVANCED

"A bug silently corrupted a subset of records over the last 6 hours. Walk through detection and recovery."

Tests whether observability (Section 6) is treated as a real capability or an afterthought. Detection depends entirely on what was instrumented *before* the incident: without data-integrity monitoring (checksums, anomaly detection on write patterns, reconciliation jobs comparing derived aggregates against source data), silent corruption can persist far longer than an outage would, precisely because nothing looked "down." Recovery: identify the exact time window and blast radius using audit logs/event sourcing if available; restore from a point-in-time backup or replay an event log up to just before the corruption (directly the DBMS guide's Point-in-Time Recovery mechanism); communicate the incident and affected scope transparently. The strong answer explicitly names that this scenario is *why* audit logs and PITR exist, not just for the "normal" crash-recovery case.

data corruption

point-in-time recovery

reconciliation job

Trade-off questions

9.5 INTERMEDIATE

Consistency vs. availability — argue both sides for a specific feature (e.g. a "like" counter).

Arguing **for availability** (eventual consistency): a like counter being off by a few, briefly, is invisible to users and recovers on its own — there is no correctness harm in serving a slightly stale count instead of rejecting the request during a partition. Arguing **for consistency**: if "likes" gate some downstream business logic (a payout threshold, a leaderboard prize cutoff), staleness stops being cosmetic and becomes a real correctness bug. The point of this question isn't reaching one "right" answer — it's testing whether the candidate resolves the trade-off by asking what the number is actually *used for*, rather than picking a side reflexively.

consistency vs. availability

per-feature consistency reasoning

9.6 ADVANCED Cost vs. performance — when is a slower, cheaper design the right answer?

Sources place this squarely at the Staff bar: cost is a real design constraint, not an afterthought bolted on at the end. A design that shaves 50ms off p99 latency by keeping an entire dataset in an expensive in-memory store is the wrong answer if the actual product requirement tolerates 300ms and the cost difference is substantial — the strong answer explicitly ties the performance target back to Step 2's stated non-functional requirement rather than defaulting to the fastest possible option. Naming "build vs. buy" here too: operating a self-managed distributed cache cluster has a real engineering-time cost that a managed service trades for a per-request price — worth stating as a deliberate choice, not an oversight.

cost as a design constraint

build vs. buy

9.7 INTERMEDIATE Latency vs. throughput — why can optimizing for one hurt the other?

Batching requests together (grouping many small writes into one larger write) improves **throughput** (more total work done per second) but directly hurts **latency** for any individual request, which must now wait for the batch window to fill before being processed. Conversely, processing every request the instant it arrives minimizes per-request latency but caps throughput lower, since per-request overhead (connection setup, lock acquisition) is paid every single time rather than amortized. A system's actual requirement (interactive user-facing request vs. a background bulk-ingestion pipeline) should decide which side of this dial to favor — this is the same batching trade-off TCP's Nagle's algorithm and Kafka's producer batching both make, at different layers.

batching

latency vs. throughput

9.8 ADVANCED

"p99 latency has crept up 3x over the last two weeks, with no code deploys in that window. Diagnose."

The absence of a deploy in the window rules out "a bad release" as the cause and points toward a *gradually accumulating* condition — the diagnostic order: check whether traffic volume has also grown over the same window (organic growth outpacing current capacity, Section 9.1's slower cousin); check database query plans and index health (a table that's grown past where an index remains effective, or degraded statistics misleading the query planner — directly the DBMS guide's stale-statistics failure mode); check for a slow memory leak gradually increasing GC pause frequency (the JS guide's memory-leak diagnostic applies at the service level too); check whether a background job's growing dataset size is competing for the same resources as the live request path. The unifying skill: matching the *shape* of the symptom (gradual, not a step change) to causes that are themselves gradual.

gradual degradation diagnosis

query plan degradation

9.9 ADVANCED

"Error rate is fine, but users are reporting the feature 'feels broken.' What do you check?"

Tests whether golden signals (Section 6.1) are understood as a set, not individually sufficient. A healthy error rate says nothing about **latency** (a request that eventually succeeds after 8 seconds reads as "broken" to a user even though it's not counted as an error) or about a client-side/UX issue entirely outside the service's own metrics (a frontend bug misrendering a correct response). The diagnostic path: check p99/p999 latency specifically, not just the average (a small percentage of very slow requests can be invisible in an average while still being what most complaint-filing users actually experienced); check whether the complaint correlates with a specific client version, region, or user segment rather than being uniform.

tail latency vs. average

golden signals as a set

WHAT THE WEB SOURCES EMPHASIZE

Multiple current sources explicitly state that pushing a design to "10x, 100x, 1000x the initial requirements... is the single most consistent evaluation criterion" — this section's scenario questions (9.1–9.4) are built directly around that scaling-pressure-test pattern rather than static, one-shot prompts.

10 Case Study: URL Shortener

FREQUENCY SIGNAL: NAMED DIRECTLY AS "ONE OF THE MOST COMMON SYSTEM DESIGN INTERVIEW QUESTIONS"; BYTEBYTEGO, HELLO INTERVIEW, AND MULTIPLE AGGREGATED QUESTION LISTS ALL PLACE IT AT OR NEAR THE TOP OF THE FREQUENCY LIST — OFTEN THE FIRST HLD PROMPT A CANDIDATE EVER RECEIVES

Problem statement, as asked

"Design a URL shortening service like TinyURL/Bitly: given a long URL, return a short one; given the short one, redirect to the original."

Clarifying questions: Custom aliases supported? Expiration/TTL on links? Analytics (click counts) required? Expected scale (reads vastly outnumber writes for this system — worth confirming explicitly)?

Functional & non-functional requirements

- **FR1** — Shorten a long URL into a unique short code.
- **FR2** — Redirect a short code to its original URL (302).
- **FR3** — Optional: custom alias, expiration, click analytics.
- **NFR1** — Read-heavy: redirects vastly outnumber creations (a realistic assumption to state and estimate against).
- **NFR2** — Redirect latency must be very low — this is on the critical path of every single click through the service.
- **NFR3** — Short codes must be effectively unique with no collisions.

Capacity estimation (Step 3 of the framework, worked)

Assume: 100M new short URLs/month, 100:1 read:write ratio

Write QPS	= 100,000,000 / (30 × 86,400)	≈ 39 QPS (avg)
Read QPS	= 39 × 100	≈ 3,900 QPS (avg)
Peak reads	= 3,900 × 3 (peak multiplier)	≈ 11,700 QPS

Storage/yr	= 100M × 12 × 500 bytes/record	≈ 600 GB/year
→ over 5 years with replication factor 3		≈ 9 TB total — comfortably within a sharded KV store

The takeaway to state explicitly: this is a **read-heavy, cache-friendly, storage-light** workload — that single sentence, derived from the numbers above, should drive every subsequent architecture choice (aggressive caching, no exotic storage needed).

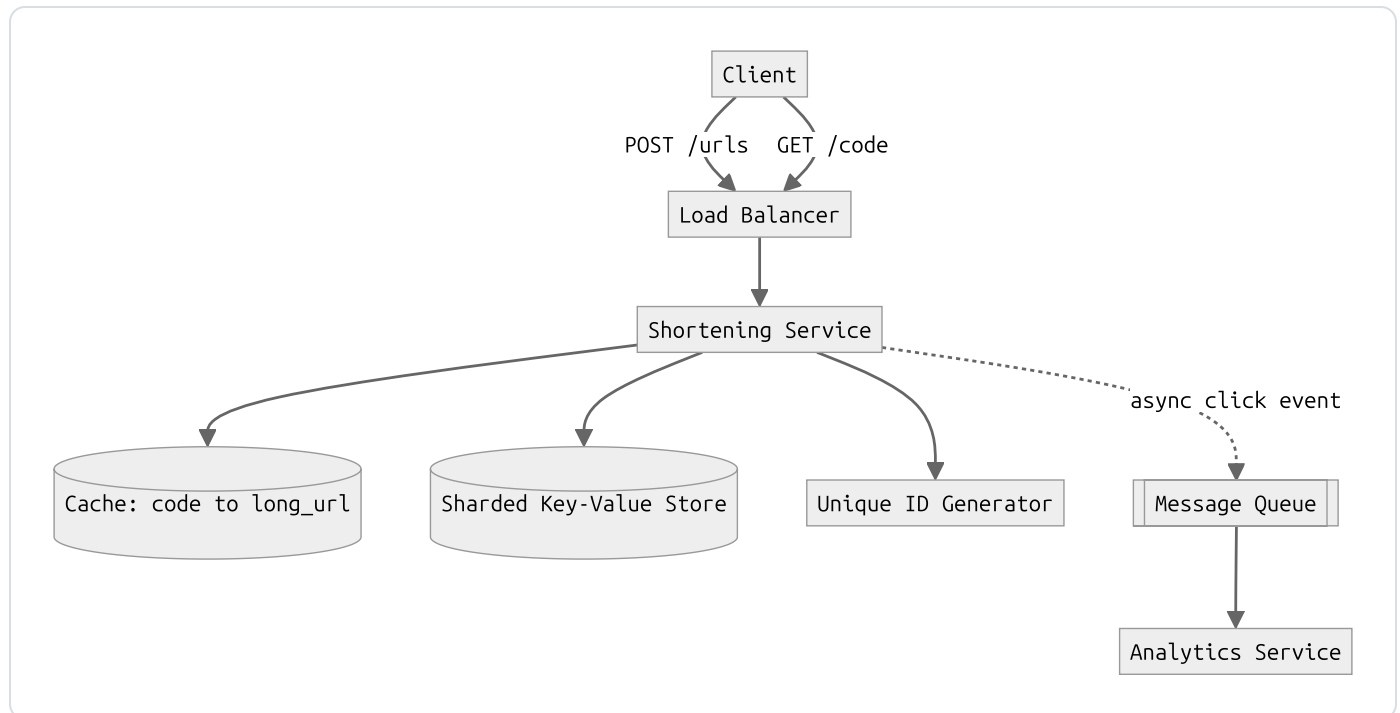
API design

See Section 2.3 for the full endpoint set (already worked as this case study's own example) — POST /api/v1/urls, GET /{shortCode} (302), GET /api/v1/urls/{shortCode}/stats.

Data model

```
urls {  
  short_code  VARCHAR(7)  PRIMARY KEY  // base62, ~57B combinations at 7 chars  
  long_url    TEXT        NOT NULL  
  created_at  TIMESTAMP  
  expires_at  TIMESTAMP  NULL  
  click_count BIGINT      DEFAULT 0    // or a separate analytics store, see below  
}
```

High-level architecture



Key request flow: shortening a URL

1. Client POSTs the long URL; the service requests a unique ID from the **ID generator** (below).
2. The ID is encoded to base62 to produce a short, URL-safe code.
3. The (code, long_url) pair is written to the sharded store, and the service returns the short URL.

Key request flow: redirecting

1. Client GETs the short code; the service checks the **cache** first (Section 4.1, cache-aside).
2. On a hit, issue the 302 immediately — this is the overwhelmingly common path given NFR1's read-heavy profile.
3. On a miss, read from the sharded store, populate the cache, then redirect.

4. The click event is fired *asynchronously* onto a queue (Section 5) rather than synchronously incrementing a counter on the critical path — the redirect must never wait on analytics bookkeeping.

Unique ID generation — the core design decision

APPROACH	MECHANISM	TRADE-OFF
Random + collision check	Generate a random code, check the DB for a collision, retry if taken	Simple; collision probability rises as the namespace fills, adding retries under load
Base62-encoded auto-increment	A single global counter, base62-encoded	Guaranteed unique, no collisions; the counter itself is a single point of contention/failure unless partitioned
Pre-generated key ranges (Ticket Server pattern)	Each service instance is handed a distinct range of IDs upfront (e.g. instance A gets 1–1M, instance B gets 1,000,001–2M)	No contention per request, no coordination needed at write time; wastes unused IDs if an instance dies mid-range

The pre-generated key-range approach is the answer most current sources converge on for exactly this problem — it removes the ID generator as a synchronous bottleneck on every single write, trading a small amount of ID-space waste for genuine horizontal scalability of the write path.

Scaling strategy

- **Caching** — per NFR1, an aggressive cache-aside layer in front of the datastore absorbs the overwhelming majority of read traffic; a CDN/edge cache (Section 1.1) can cache redirects for very popular links even closer to the user.
- **Sharding** — the key-value store shards by `short_code`'s hash (consistent hashing, Section 1.3) — this key is, by construction, uniformly distributed, avoiding Section 3.4's hot-shard trap entirely.

Reliability & disaster recovery

The redirect path is the one that must survive any single component failure — a cache node failure should degrade to a slower DB read, never a failed redirect. The datastore is replicated (Section 3) across availability zones at minimum; given the read-heavy, tolerant-of-eventual-consistency profile, asynchronous cross-region replication (Section 6.4) is an acceptable, cost-effective DR posture rather than costly synchronous multi-region writes.

Consistency model

The (code → long_url) mapping is **immutable once created** — this sidesteps most consistency concerns entirely, since there's no concurrent-update conflict to resolve. Click-count analytics are explicitly eventually consistent (Section 3.2) — a delayed or approximate count is invisible to the end user and costs nothing in correctness.

Security considerations

- **Rate limiting** creation requests per IP/API key (Section 13's case study, reused directly) — otherwise the service becomes a free, abusable URL-obfuscation tool for spam/phishing.
 - **Malicious URL screening** — checking submitted long URLs against a known-malware/phishing blocklist before shortening, since the service is otherwise a convenient way to hide a malicious destination behind a trusted-looking short domain.
-

Observability

Golden signals (Section 6.1) on the redirect path specifically — this is the path with the tightest latency budget and the highest request volume, so its p99 and error rate deserve their own dedicated dashboard and SLO, separate from the lower-volume creation path.

Bottlenecks & mitigations

- **A single viral link** — a hot key (Section 9.3) on one cache node/shard; mitigated by the edge-CDN layer absorbing the bulk of it before it ever reaches the origin cache.
 - **ID generator range exhaustion under a sudden write burst** — mitigated by each instance requesting its next range proactively before fully exhausting its current one, not reactively after running out.
-

Interviewer follow-ups & deep-dive questions

- **"Now support custom aliases."** — tests whether the unique-ID approach can coexist with user-chosen strings (a uniqueness check against the same store, not a separate system).
- **"What if we need this across multiple regions with low latency everywhere?"** — pushes toward the multi-region replication and DR discussion in Section 6.4.
- **"How would you delete expired links efficiently at scale?"** — tests whether the candidate reaches for a background sweep job (indexed by expires_at) rather than a per-request check alone.

What strong candidates say: "This is read-heavy, so I'm optimizing the redirect path first and treating analytics as strictly async and eventually consistent." **Red flag:** spending the majority of interview time on the shortening/hashing algorithm while never mentioning caching at all, despite the workload being overwhelmingly read-dominated.

11 Case Study: Chat System (WhatsApp)

FREQUENCY SIGNAL: NAMED DIRECTLY ACROSS HELLO INTERVIEW'S PROBLEM BREAKDOWNS, GEEKSFORGEEKS, AND MULTIPLE DEDICATED "DESIGN WHATSAPP" WALKTHROUGHS AS ONE OF THE STANDARD COMMUNICATION-SYSTEMS PROMPTS, TESTING REAL-TIME BIDIRECTIONAL DELIVERY SPECIFICALLY

Problem statement, as asked

"Design a real-time one-on-one and group messaging system like WhatsApp — message delivery, online presence, and delivery/read receipts."

Clarifying questions: One-on-one only, or group chats too? Media (images/video) in scope, or text only? Message history retention — forever, or a bounded window? End-to-end encryption required?

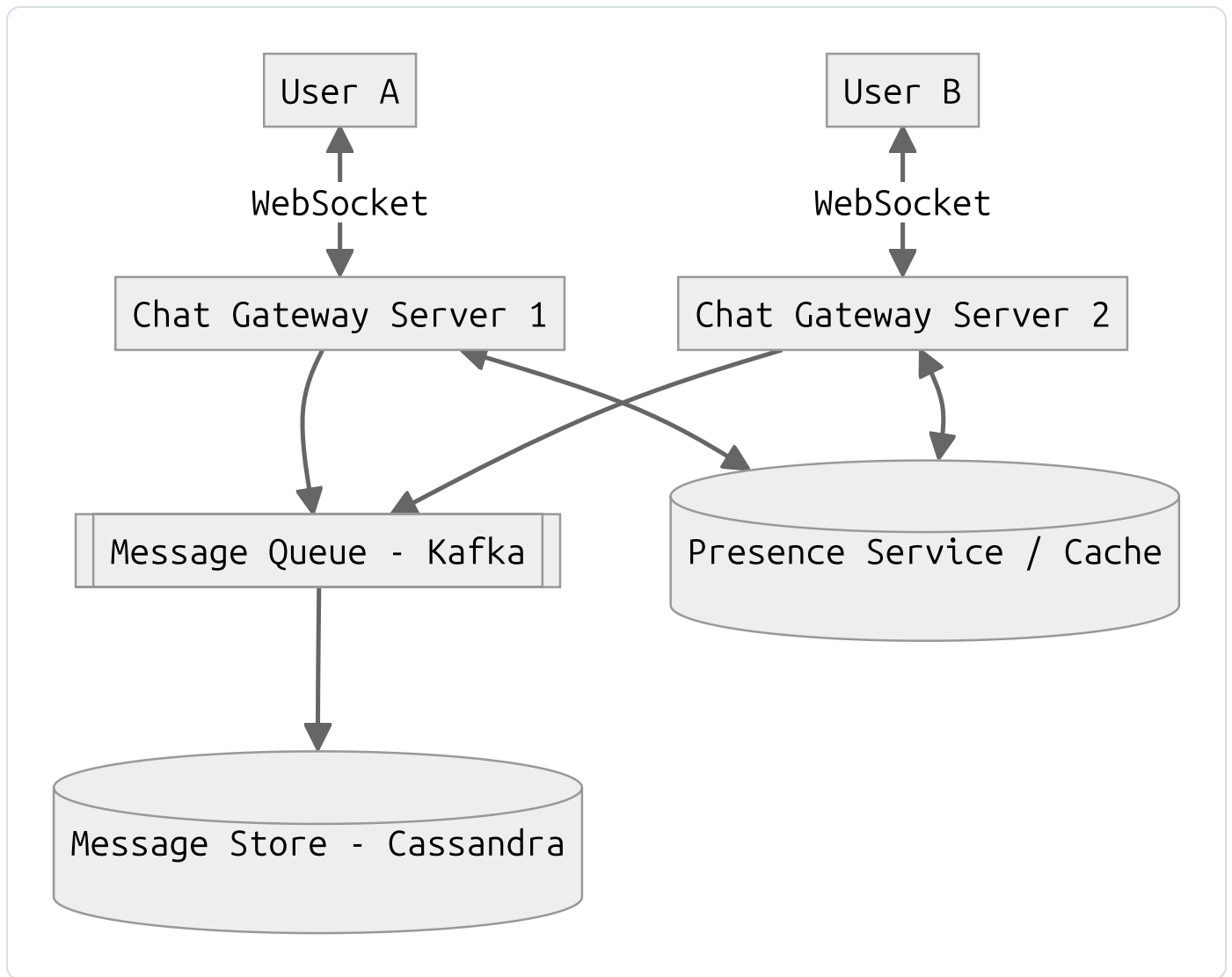
Functional & non-functional requirements

- **FR1** — Send/receive messages in real time between two users.
- **FR2** — Group messaging (fan-out to N members).
- **FR3** — Delivery and read receipts; online/last-seen presence.
- **FR4** — Message history persists and syncs across a user's multiple devices.
- **NFR1** — Low latency delivery (near-instant, sub-second) when the recipient is online.
- **NFR2** — Messages must not be lost, even if the recipient is offline when sent.
- **NFR3** — Massive concurrent connection scale (hundreds of millions of simultaneously open connections).

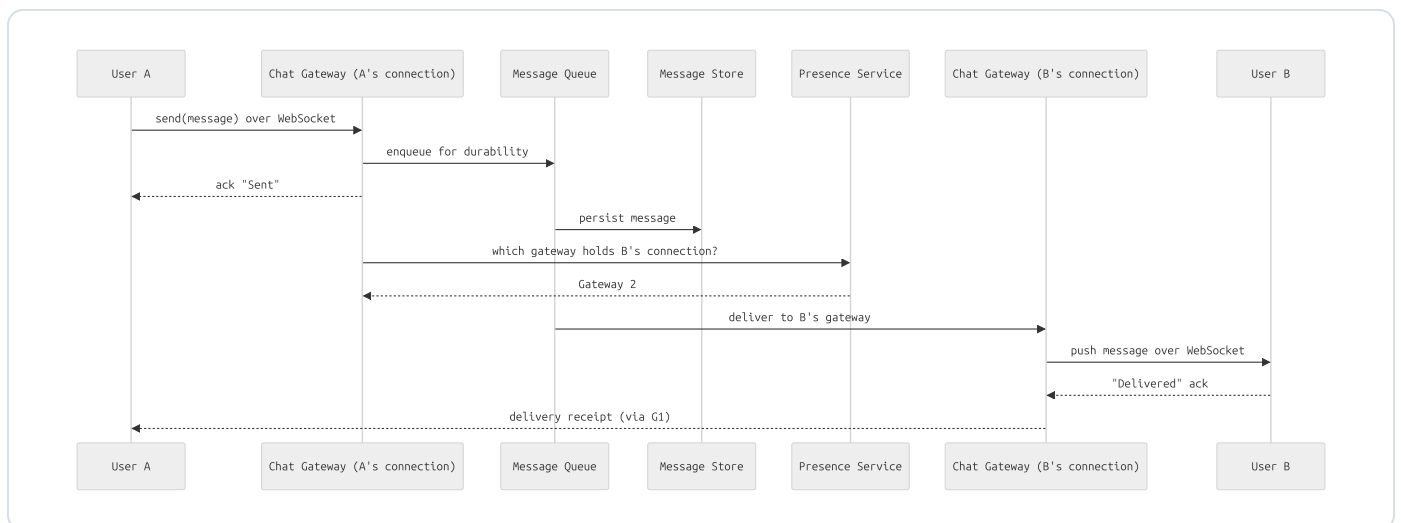
Why WebSockets, not polling

Per current sources, the expected interview approach: name **WebSockets** as the primary transport (a persistent, full-duplex connection matching chat's inherently bidirectional nature, avoiding the latency and overhead of repeated polling), with **long polling** named explicitly as the fallback for clients/networks that can't hold a WebSocket open. Stating both, and why WebSockets wins for this specific workload, is the expected depth — not just "use WebSockets" with no comparison.

High-level architecture



Key request flow: User A sends a message to User B



Per current sources, the sequence is: the gateway authenticates and *immediately* queues the message for durability before doing anything else — this ordering matters, since it's what

guarantees NFR2 (no message loss) even if the recipient's gateway or connection fails downstream of this point. The sender gets an immediate "Sent" acknowledgment from queuing alone, decoupled from actual delivery.

Data model

```
messages {
  message_id    UUID          PRIMARY KEY
  conversation_id UUID        // partition key – all messages for one conversation co-located
  sender_id     UUID
  content       BLOB          // encrypted payload
  sent_at       TIMESTAMP
  status        ENUM(sent, delivered, read)
}
// Cassandra/DynamoDB: high write throughput, simple query pattern
// (fetch messages for one conversation_id, time-ordered) – a strong fit per Section 3.3
```

Scaling strategy

- **Connection scaling** — many gateway servers, each holding a subset of live WebSocket connections; a **presence service** (backed by a fast cache) tracks which gateway instance holds which user's connection, so any gateway can route an outbound message to the right one.
 - **Group messages** — fan-out to N recipients' respective gateways via the message queue's pub-sub capability (Section 5.1) — the queue naturally handles "deliver this one message to potentially hundreds of group members" without the sending gateway needing direct knowledge of every recipient's connection.
 - **Storage** — partitioned by `conversation_id` so a single conversation's full history is efficiently retrievable in one range query, directly informed by the access pattern (Step 5 of the framework).
-

Reliability & disaster recovery

An offline recipient's messages remain queued/persisted (NFR2) and are delivered on reconnect — the presence service marking a user offline should trigger the gateway to hold rather than drop undeliverable pushes. A chat gateway instance crashing should not lose in-flight messages, since they were already durably queued before the crash (the same ordering guarantee named above) — clients reconnect to a different gateway instance transparently, with the presence service updated accordingly.

Consistency model

Message ordering *within one conversation* must be preserved (a queue partitioned by `conversation_id`, Section 5.2, guarantees this) — but strict global ordering across all conversations system-wide is unnecessary and would be a costly, pointless constraint. Delivery/read receipts are eventually consistent by nature (a receipt is inherently a later, separate event from the message itself).

Security considerations

End-to-end encryption (if in scope) means the server stores and routes only ciphertext — it never has the plaintext content or the ability to decrypt it, which is the specific privacy guarantee that differentiates this from server-side encryption-at-rest alone. Connections authenticate via short-lived tokens (Section 7.1) at WebSocket handshake time, not just at initial login.

Observability

Golden signals per gateway instance (connection count as a saturation signal specifically, alongside the standard four) plus an end-to-end delivery-latency metric (time from sender's "Sent" to recipient's "Delivered") as the single most user-visible health indicator for this system.

Bottlenecks & mitigations

- **The presence service itself becoming a bottleneck** at NFR3's scale — mitigated by sharding presence data the same way the connections themselves are distributed, rather than one global lookup table.
 - **Group chats with very large membership** — the same celebrity-fanout problem as the News Feed case study (Section 12); very large groups may warrant a pull-based read model rather than pushing to every member's gateway individually.
-

Interviewer follow-ups & deep-dive questions

- **"Support message editing and deletion."** — tests whether the immutable-append-only assumption in the data model was stated explicitly, and how a mutable event is layered on top without breaking ordering.
- **"Now scale to a group of 100,000 members (a broadcast channel)."** — directly the celebrity-fanout problem; expects the fanout-on-read pivot from Section 12.
- **"How do you sync message history to a new device a user just logged into?"** — tests the data model's ability to serve a full historical range query per conversation, not just live delivery.

What strong candidates say: "I'm queuing for durability before attempting delivery, so a gateway crash between those two steps never loses the message." **Red flag:** designing delivery as

a direct, synchronous gateway-to-gateway call with no durable queue in between — this silently violates NFR2 the moment either gateway is unavailable.

12 Case Study: News Feed (Twitter)

FREQUENCY SIGNAL: NAMED DIRECTLY ALONGSIDE URL SHORTENER AS ONE OF THE STANDARD CASE STUDIES ACROSS BYTEBYTEGO, GEEKSFORGEEKS, AND MULTIPLE DEDICATED "DESIGN TWITTER/NEWS FEED" WALKTHROUGHS – SPECIFICALLY VALUED FOR TESTING THE PUSH VS. PULL FANOUT TRADE-OFF AND ITS "CELEBRITY PROBLEM" EDGE CASE

Problem statement, as asked

"Design a news feed system like Twitter/Instagram — users follow other users, post content, and see a reverse-chronological (or ranked) feed of posts from people they follow."

Clarifying questions: Reverse-chronological, or ranked/algorithmic? Media posts in scope? Typical follower-count distribution — are we designing for a small social graph, or one with celebrity accounts with tens of millions of followers (this directly determines whether the celebrity problem is in scope)?

Functional & non-functional requirements

- **FR1** — Post content; follow/unfollow other users.
- **FR2** — Retrieve a feed of recent posts from followed accounts.
- **NFR1** — Feed reads must be fast — this is the most frequently hit endpoint in the entire system.
- **NFR2** — Handle a highly skewed follower-count distribution (most accounts have few followers; a small number have tens of millions).

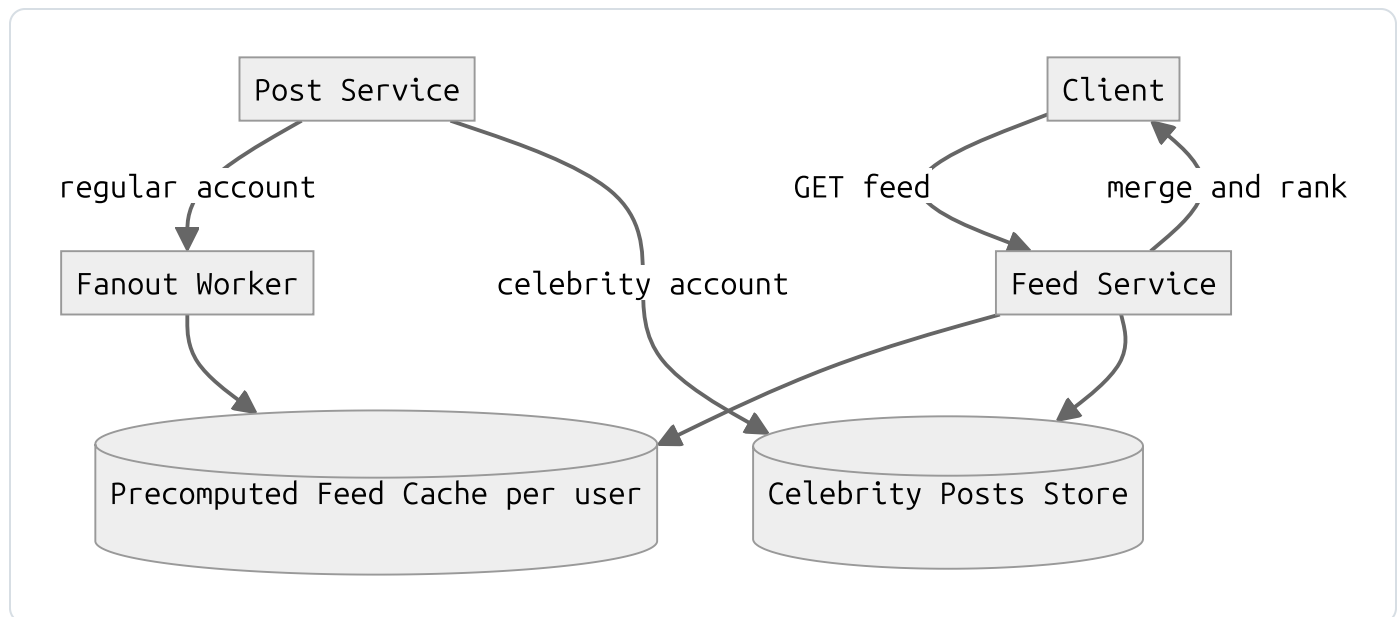
The core design decision: fanout-on-write vs. fanout-on-read

	FANOUT-ON-WRITE (PUSH)	FANOUT-ON-READ (PULL)
Mechanism	On post, immediately write it into every follower's precomputed feed cache	On feed request, query and merge posts from every followed account in real time
Read cost	Very cheap — feed is already assembled	Expensive — a fan-in query across every followed account, every request
Write cost	Expensive, proportional to follower count	Cheap — one write, regardless of follower count
Breaks down when	An account has millions of followers — "the celebrity problem"	A user follows a very large number of accounts

The celebrity problem, and the hybrid fix

Per current sources: pushing a single celebrity's post into millions of followers' cached feeds wastes enormous write capacity for a post most of those followers may never even scroll to. The standard, explicitly named hybrid solution: **fanout-on-write for regular accounts** (below some follower-count threshold, e.g. ~10,000) — write amplification stays manageable; **no fanout for celebrity accounts** above that threshold — their posts are stored once and fetched live. A follower's feed request then **merges** their precomputed (fanout-on-write) feed with a real-time fetch of posts from any celebrity accounts they follow, combining both models rather than forcing one to cover every case.

High-level architecture



Key request flow: a regular user posts

1. Post Service writes the post to durable storage and publishes a "new post" event to a queue (Section 5).
2. A **fanout worker** consumes the event, looks up the poster's follower list, and asynchronously writes the post into each follower's precomputed feed cache — asynchronous specifically so the poster's own request isn't held up by however many followers they have.

Data model

```
posts { post_id PK, author_id, content, created_at }
follows { follower_id, followee_id }           // the social graph
feed_cache[user_id] -> [post_id, post_id, ...] // precomputed, capped length (e.g. last 800 posts)
```

Scaling strategy

The feed cache is sharded by `user_id` (Section 3.4) — a naturally well-distributed key, since no single user's feed is disproportionately hot the way a celebrity's *post* is. This is the same hot-key/hot-partition concept from Section 9.3, resolved here architecturally by routing celebrities down an entirely different path rather than attempting to spread their load after the fact.

Reliability & disaster recovery

Fanout is explicitly asynchronous and best-effort against a strict deadline — a fanout worker falling behind under load degrades to "some followers see the post a few seconds later," not a failed post. The source of truth is the durably stored post itself; the feed cache is a derived, rebuildable projection, so losing it entirely is a performance incident, not a data-loss incident — worth stating explicitly as a reliability property of the design.

Consistency model

Feeds are eventually consistent by design (Section 3.2) — a post appearing in a follower's feed a few seconds after being published is invisible/acceptable to users, which is exactly the property that makes the fully-asynchronous fanout architecture above acceptable in the first place.

Observability

Fanout lag (time from post creation to fanout completion across all followers) is the single most important custom metric for this system — a golden-signals dashboard alone won't surface a fanout worker falling behind, since that failure mode doesn't manifest as request errors on the read path.

Bottlenecks & mitigations

- **Celebrity posts, unmitigated** — the entire reason the hybrid model exists; without it, the write-amplification problem is the single dominant bottleneck of the whole system.
- **A user following an unusually large number of celebrities** — the merge step in feed reads now has to fan-in across many celebrity stores; mitigated by capping how many real-time sources are merged per request and caching the merge result briefly.

Interviewer follow-ups & deep-dive questions

- **"Rank the feed instead of showing it reverse-chronologically."** — introduces a scoring/ranking service between the merge step and the client; tests whether the candidate keeps ranking as a separable concern layered on top of the existing fanout architecture, not a rewrite of it.
- **"What's your threshold for 'celebrity,' and how do you handle an account crossing it?"** — tests whether the candidate treats the threshold as a tunable, monitored parameter rather than a fixed constant, and has a migration plan for an account whose follower count grows past it.
- **"How would you backfill a brand-new user's feed the moment they follow 50 accounts at once?"** — a cold-start problem distinct from steady-state fanout, testing whether the candidate's design has an on-demand bulk-generation path in addition to incremental fanout.

What strong candidates say: "I'm using a hybrid model — most accounts get pushed, celebrity accounts get pulled — because the write cost of pushing to a celebrity's follower count would dominate the system otherwise." **Red flag:** proposing pure fanout-on-write with no mention of the celebrity case at all — this is specifically the gap current sources report interviewers probe for directly.

13 Case Study: Distributed Rate Limiter

FREQUENCY SIGNAL: NAMED DIRECTLY AMONG THE MOST COMMON SYSTEM DESIGN PROMPTS, SPECIFICALLY THE HLD COUNTERPART TO THE SINGLE-PROCESS RATE LIMITER — THE INTERESTING CONTENT HERE IS ENTIRELY ABOUT SHARED STATE ACROSS A FLEET, WHICH A SINGLE-MACHINE DESIGN (AS IN THE COMPANION LLD GUIDE) DOESN'T NEED TO ADDRESS AT ALL

Problem statement, as asked

"Design a rate limiter for an API platform that enforces per-client limits (e.g. 100 requests/minute) across a fleet of many stateless API servers behind a load balancer."

Clarifying questions: Per-API-key, per-user, or per-IP limiting? A single global limit, or different limits per subscription tier? Is a small amount of limit-enforcement imprecision under extreme load acceptable in exchange for lower latency?

Functional & non-functional requirements

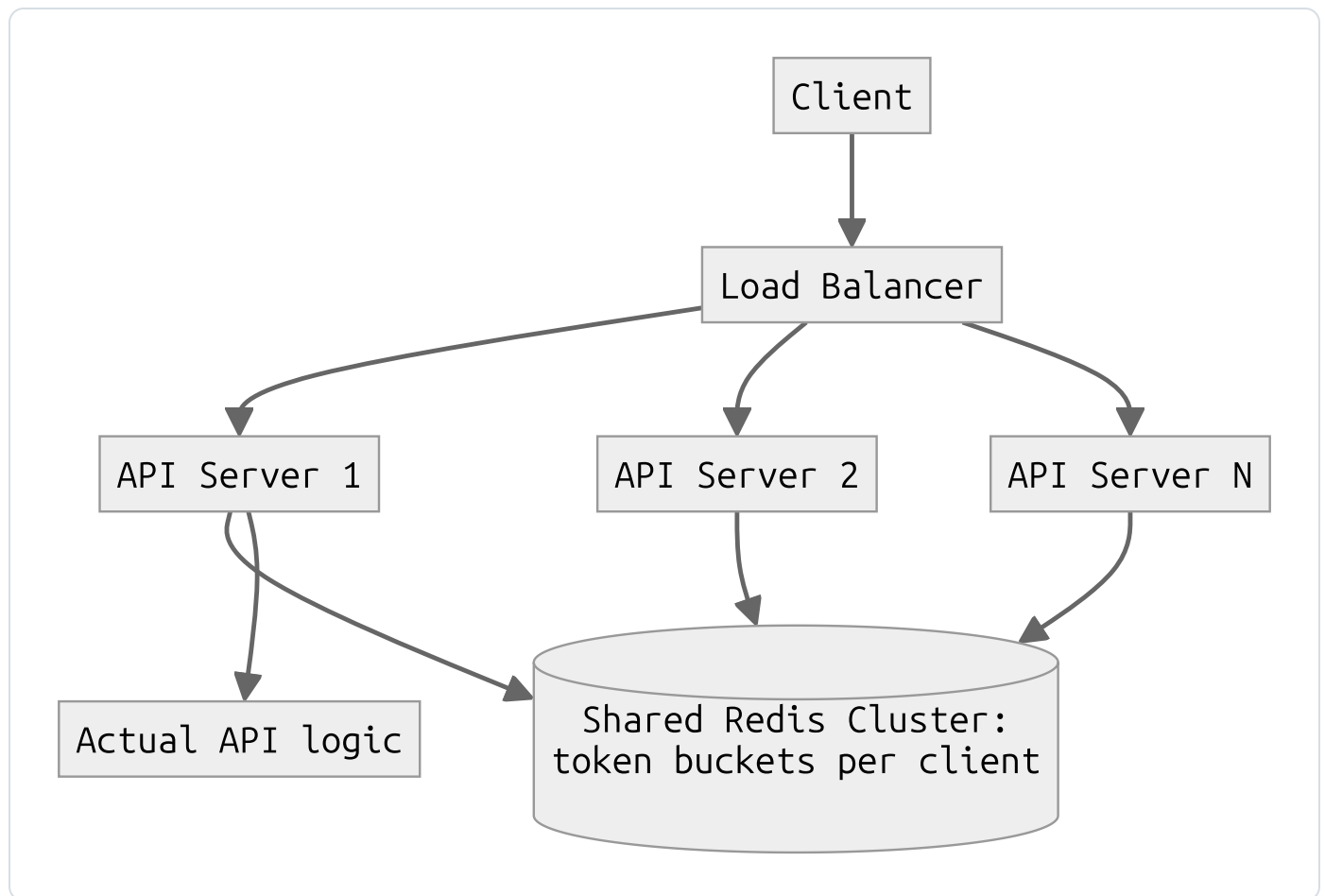
- **FR1** — Reject requests beyond a client's configured limit within a time window, returning HTTP 429.
- **FR2** — Support different limits per client tier.
- **NFR1** — The limiter check itself must add negligible latency — it runs on every single request across the platform.

- **NFR2** — Correct *even though* many stateless API server instances are checking the same client's limit concurrently — this is the entire problem the LLD version of this case study doesn't have to solve.

Why this can't just be Section 4's in-memory Token Bucket, unmodified

A per-process in-memory token bucket (as built in the companion LLD guide) works perfectly for one process — but this system has *many* stateless API server instances (Section 1.4) behind a load balancer, and a client's requests can land on any of them, in any order, across any instance. If each instance kept its own independent in-memory bucket, a client could get `limit × number_of_instances` requests through — one full limit's worth per instance — completely defeating the point of a shared, platform-wide limit. The fix: move the counter state *out* of any single process into a shared, external store every instance reads and writes.

High-level architecture



Key request flow

1. A request for client X arrives at any API server instance (routed arbitrarily by the load balancer).
 2. That instance executes an **atomic** "check and decrement" operation against client X's shared token count in Redis (a Lua script or Redis's native atomic INCR+TTL combination) — atomicity here is the entire correctness requirement, since two instances could otherwise both read "1 token left" and both allow the request, exceeding the limit.
 3. If allowed, the request proceeds to the actual API logic; if not, the server returns **429** immediately, with a Retry-After header.
-

Data model

```
// Redis key per client, holding current token count + last-refill timestamp
// TTL set to the window length, so idle clients' state expires automatically
key: "ratelimit:{client_id}"
value: { tokens: 42, last_refill: 1234567890 }
TTL: window_seconds
```

Scaling strategy

Redis itself is now the shared dependency every API server calls on every request — it must be able to handle platform-wide QPS, not per-client QPS. Mitigations: shard the rate-limit keyspace across a Redis Cluster (by client_id, an inherently well-distributed key per Section 3.4), and consider a small amount of relaxed precision — e.g. each API server locally caches "this client is clearly nowhere near their limit" for a very short window (tens of milliseconds) to avoid a Redis round-trip on every single request, falling back to the authoritative shared check only when a client is close to their limit.

Reliability & disaster recovery

This is a case where **failing open vs. failing closed** is an explicit, storable design decision: if the shared Redis cluster becomes unavailable, does the platform stop enforcing limits entirely (fail open — availability preserved, abuse risk temporarily accepted) or reject all requests (fail closed — abuse fully prevented, but a Redis outage now takes down the entire API platform)? Current sources' implicit expectation, and the generally accepted default for rate limiting specifically: **fail open** — a rate limiter existing to prevent abuse should not itself become the platform's single point of total failure.

Consistency model

The count itself only needs to be *approximately* correct under extreme concurrency — per NFR1's latency constraint, a design that's perfectly precise but adds meaningful latency to every request is very likely the wrong trade-off for this specific system. Stating this explicitly (rather than chasing perfect atomicity everywhere) is the trade-off-reasoning signal Section 15's rubric rewards.

Security considerations

The rate limiter is itself a core piece of abuse prevention (Section 7) for every other system behind it — it's frequently the first and cheapest line of defense named in this guide's other case studies (URL Shortener's creation endpoint, the Notification System's send endpoint) rather than a standalone feature.

Observability

The 429 rate itself is a golden signal worth its own dashboard, separate from the general error rate (Section 6.1) — a rising 429 rate can mean legitimate organic growth pushing against configured limits (a business/capacity-planning signal) just as easily as it can mean an abuse pattern, and distinguishing the two requires per-client breakdown, not just an aggregate count.

Bottlenecks & mitigations

The shared Redis dependency is, by construction, a new potential bottleneck and single point of failure introduced specifically to solve NFR2 — this trade-off (introducing shared infrastructure to correctly enforce a shared limit) should be named explicitly, alongside the fail-open mitigation above.

Interviewer follow-ups & deep-dive questions

- **"What if Redis itself needs to scale across regions, with clients hitting servers in different regions?"** — pushes toward either a per-region limit (simpler, slightly less precise globally) or a more complex globally-synchronized counter (Section 6.4's cross-region trade-offs apply directly).
- **"Different algorithms need different limits — some strict fixed windows, some bursty."** — validates whether the token-bucket choice was actually a deliberate one (Strategy-pattern-style swappable, per the LLD guide) rather than hardcoded.
- **"How do you avoid unbounded memory growth from millions of rarely-seen clients?"** — the TTL on each Redis key (shown in the data model above) is the direct answer; a candidate who names this unprompted is showing real production awareness.

What strong candidates say: "The interesting problem here isn't the algorithm, which is settled — it's making the count correct across many concurrent stateless servers without that shared state becoming its own single point of failure." **Red flag:** presenting the exact same in-memory single-process design from an LLD interview without addressing why it breaks the moment there's more than one server instance.

14 Case Study: Notification System

FREQUENCY SIGNAL: NAMED DIRECTLY ALONGSIDE URL SHORTENER, CHAT, AND NEWS FEED AS ONE OF "THE MOST FREQUENTLY ASKED SYSTEM DESIGN INTERVIEW PROBLEMS" — VALUED FOR TESTING THIRD-PARTY INTEGRATION, MULTI-CHANNEL FANOUT, AND IDEMPOTENT DELIVERY TOGETHER IN ONE SYSTEM

Problem statement, as asked

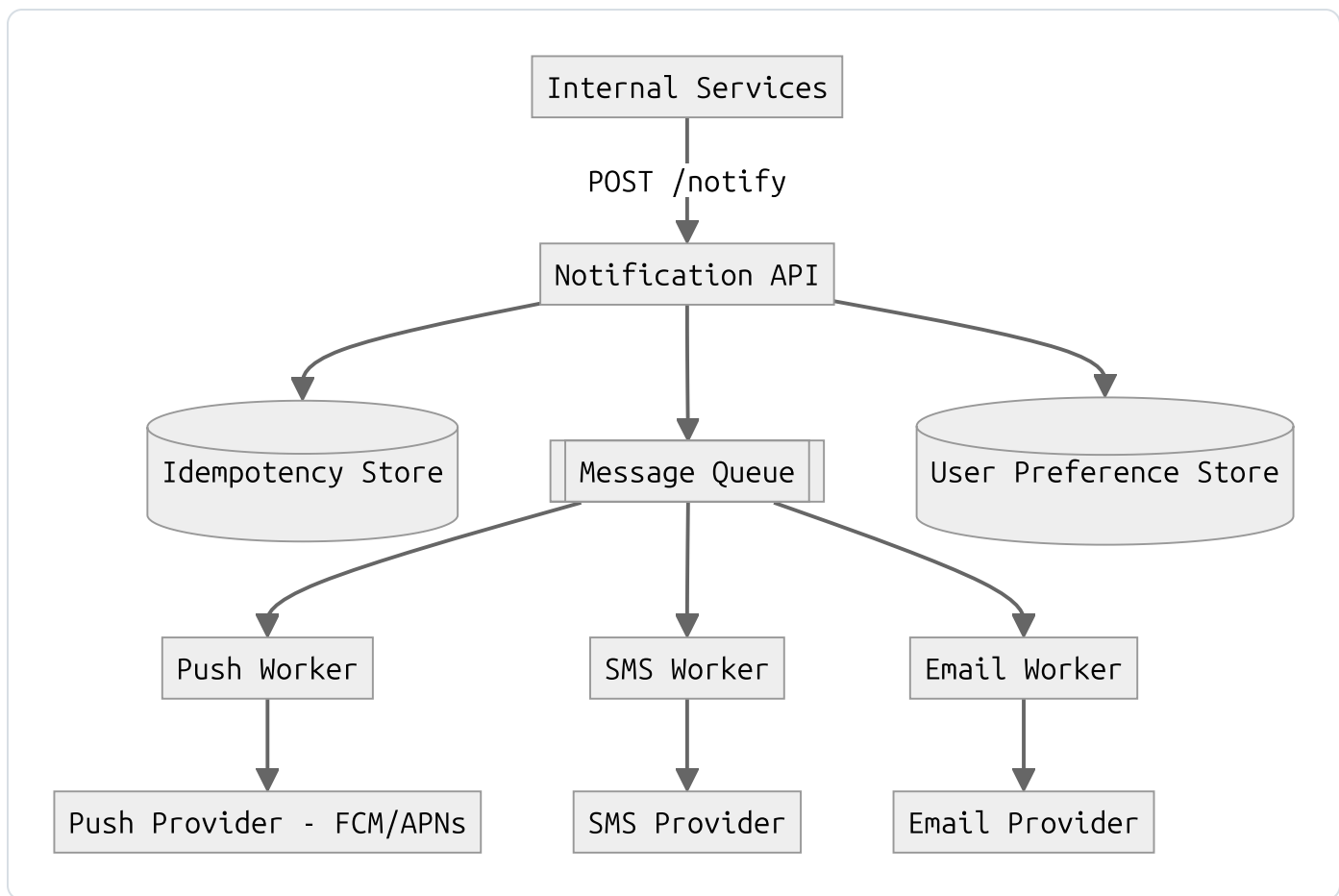
"Design a notification system that can send push notifications, SMS, and email to users, triggered by events from many other internal services."

Clarifying questions: Which channels are in scope (push/SMS/email/in-app)? Are notifications user-configurable (opt-out per channel/type)? Is exact delivery-order across channels required, or is each channel independent? What's the expected fan-out size per event (a single user's password reset vs. a promotional blast to millions)?

Functional & non-functional requirements

- **FR1** — Accept a notification request from any internal service and deliver it via the appropriate channel(s).
- **FR2** — Respect user notification preferences (opted out of marketing SMS, etc.).
- **FR3** — Support both single-recipient (a password-reset email) and bulk (a promotional push to millions) sends.
- **NFR1** — At-least-once delivery — a lost password-reset notification is a real user-facing failure.
- **NFR2** — Must not double-send the same logical notification if an upstream service retries its request.
- **NFR3** — A slow or failing third-party provider (an SMS gateway having an outage) must not block or slow down other channels/notifications.

High-level architecture



Key request flow

1. An internal service (e.g. the auth service, on password reset) calls the Notification API with an **idempotency key** (Section 5.3/2.3) unique to this logical event.
2. The API checks the idempotency store — if this key was already processed, it returns the prior result immediately without re-sending anything (directly preventing NFR2's double-send risk on upstream retry).
3. The API checks the **user preference store** to determine which channels this user actually wants for this notification type, then publishes one message per applicable channel onto separate, **per-channel queues**.
4. Each channel has its own dedicated worker pool calling its own third-party provider — this separation is exactly what satisfies NFR3: an SMS provider outage backs up only the SMS queue, leaving push and email workers completely unaffected.

Data model

```
notifications { notification_id PK, idempotency_key UNIQUE, user_id, type, status, created_at }
user_preferences { user_id, channel, notification_type, opted_in BOOLEAN }
delivery_attempts { notification_id, channel, attempt_number, status, provider_response }
```

Scaling strategy

Per-channel queues (rather than one shared queue) is the core scaling decision — it lets each channel's worker pool scale independently based on that specific provider's actual throughput limits, and directly delivers NFR3's isolation requirement. Bulk sends (a promotional blast to millions) are enqueued as a batch job that fans out onto the same per-channel queues gradually, rather than all at once — protecting the third-party providers' own rate limits (Section 13's rate limiter applies here too, on the outbound side).

Reliability & disaster recovery

Failed provider calls retry with exponential backoff (Section 5.3), and a message that exhausts its retries routes to a dead-letter queue for investigation rather than being silently dropped — for a password-reset notification specifically, a DLQ entry should page a human, not just log quietly, given the user-facing severity. Each provider integration should have a circuit breaker (the LLD guide's reliability pattern, reused directly) so a fully-down SMS provider doesn't waste worker capacity on calls that are certain to fail.

Consistency model

Delivery status (sent/delivered/failed) is inherently eventually consistent — it's only known after the third-party provider responds, sometimes asynchronously via the provider's own webhook. The idempotency check itself, however, needs to be effectively atomic (a compare-and-set against the idempotency store) to actually prevent the double-send race NFR2 requires — a plain "check then insert" without atomicity reintroduces the exact check-then-act race condition covered in the LLD guide's concurrency section.

Security considerations

The Notification API is an internal-facing endpoint, but should still authenticate calling services (Section 7.2's zero-trust principle — "internal" doesn't mean "unauthenticated") to prevent one compromised internal service from sending arbitrary notifications as any user. Provider credentials (SMS/email/push API keys) are secrets that must never be logged or embedded in delivery-attempt records that might be exposed for debugging.

Observability

Per-channel, per-provider golden signals (Section 6.1) — a provider-specific error-rate spike is the primary signal that a third-party dependency is degrading, and should trigger the corresponding

channel's circuit breaker before it's even escalated to a human.

Bottlenecks & mitigations

- **A single third-party provider outage** — mitigated structurally by the per-channel queue isolation named above, and optionally a secondary backup provider per channel for critical notification types.
- **A very large bulk send competing with time-sensitive individual notifications** (a password reset stuck behind a million-recipient promotional blast in the same queue) — mitigated by priority queues or entirely separate queues for transactional vs. bulk/marketing notifications.

Interviewer follow-ups & deep-dive questions

- **"A user should never get more than 3 marketing pushes per day, across all triggering services."** — tests whether rate limiting (Section 13) can be applied per-user, per-notification-category, layered on top of the existing per-provider rate limiting.
- **"Guarantee transactional notifications (password resets) are never delayed behind bulk sends."** — the priority-queue mitigation above, made explicit as the expected answer.
- **"How do you know if a push notification was actually seen, not just delivered to the device?"** — tests whether the candidate distinguishes provider-acknowledged delivery from client-side read receipts, which require an entirely separate client-to-server event, not something the provider can report.

What strong candidates say: "I'm isolating each channel behind its own queue and circuit breaker specifically so a dead SMS provider can't back up push or email." **Red flag:** a single shared queue and worker pool across all channels, with no mention of what happens when exactly one provider starts failing.

15 Evaluation Rubric & Leveling Guide

Synthesized directly from current sources' own stated rubric dimensions: **Problem Navigation, Solution Design, Technical Excellence, and Communication & Collaboration** — plus the specific behavioral signals that separate levels.

Mid-level: "good" looks like

- Asks clarifying questions before designing
- Produces a coherent architecture satisfying stated requirements

Senior: "good" adds

- Surfaces failure modes and trade-offs *unprompted*, not just when asked
- Pushes their own design under 10x/100x load without being told to

Staff/Principal: "good" adds

- Reasons about cost, build-vs-buy, and org/team boundaries, not just the diagram
- Considers migration strategy for an existing system, not just

- Names a sensible data store and caching choice, with a reason
- Can explain their own diagram when asked
- Handles the interviewer's first follow-up without derailing

- Discusses monitoring, alerting, deployment/rollback proactively
- Leads the conversation rather than waiting passively for direction
- Recovers gracefully when the interviewer changes a requirement mid-design

- greenfield design
- Names long-term technical-strategy implications of today's choices
- A clean architecture *alone*, with no operational/cost reasoning, is a downlevel signal at this bar

Handling ambiguity and changing requirements

Current sources are explicit that scale is "the single most consistent evaluation criterion" — interviewers routinely push a design to 10x, 100x, 1000x the original stated scale specifically to watch how the architecture (and the candidate) responds. The strong pattern: name which specific component breaks first at the new scale, and what changes to fix it (not a full redesign from scratch) — this is a direct, repeated rehearsal of the "10x/100x push" pattern baked into Sections 10–14's own follow-up lists. A requirement changing mid-design (e.g. "actually, this needs to support offline mode too") should be met with a brief acknowledgment of what part of the existing design survives unchanged and what specifically needs to adapt — not panic, and not silently ignoring the new constraint.

Communication signals interviewers watch for

- **Driving, not waiting** — proactively stating "here's what I'd tackle next" rather than pausing after each step for explicit permission to continue.
- **Narrating trade-offs, not just conclusions** — "I chose X over Y because Z" is gradable; "I chose X" alone is not, since the interviewer can't distinguish a reasoned choice from a guess.
- **Asking, not assuming, when genuinely ambiguous** — but not over-asking on points that don't actually change the design, which reads as stalling.

Why candidates actually fail, and how to recover

Per current sources, the most common failure mode isn't a bad architecture — it's a **sunk-cost reaction to pushback** (Section 0): digging in on an original choice instead of reasoning through the interviewer's challenge out loud. The second most-cited failure mode is going deep on one component before establishing the overall shape of the system, running out of time with an incomplete picture the interviewer never got to see. Both are recoverable *in the moment* they're noticed: naming the trade-off explicitly ("you're right that this creates a bottleneck at scale — here's the alternative and what it costs instead") turns a defensive moment into exactly the

collaborative signal Section 0 describes; and explicitly stepping back to sketch the remaining unaddressed components, even briefly, before time runs out, is far stronger than a fully-detailed single component with everything else left as an unlabeled box.

WHAT THE WEB SOURCES EMPHASIZE

Multiple current sources independently converge on the same four rubric dimensions and the same specific downleveling signal — a clean architecture with no proactive operational/failure-mode discussion reading as a Mid-level answer even when produced by a Senior-track candidate. Separately, sources on interview failure converge on the same root cause: "candidates rarely fail because they cannot design systems — they fail because they misunderstand the evaluation criteria," specifically underestimating how much is scored in the live conversation itself.

16 Rapid Revision, Glossary & Day-Before Checklist

A last-minute-before-the-interview pass — everything above, compressed.

THE 9-STEP FRAMEWORK, COMPRESSED

- 1. Clarify requirements + assumptions
- 2. Functional + non-functional reqs (with SLOs)
- 3. Capacity estimation (order of magnitude)
- 4. API design (external, then internal)
- 5. Data model + major entities
- 6. High-level architecture decomposition
- 7. Consistency approach per data type
- 8. Failure modes + resiliency
- 9. Observability, deployment, security, cost

DEFAULT REQUEST PATH

- Client → DNS → CDN/edge → Load Balancer/API Gateway → stateless service instances
- State lives externally (cache/DB), never in-process

CAP, PRECISELY

- Partition tolerance is mandatory, not optional
- The real choice is C-vs-A, and only *during* a partition
- Decide consistency *per data type*, not once for the whole system

CASE STUDY → CORE DESIGN DECISION

- URL Shortener — read-heavy → aggressive caching; pre-generated ID ranges avoid write contention

- Chat (WhatsApp) — WebSockets + durable queue before ack, so a gateway crash never loses a message
- News Feed — hybrid fanout: push for regular accounts, pull for celebrities
- Rate Limiter — shared external state (Redis) across stateless servers; fail open on outage
- Notifications — per-channel queues + idempotency key, so one dead provider can't block the rest

THE FOUR GOLDEN SIGNALS

- Latency, Traffic, Errors, Saturation
- SLI = measured; SLO = internal target; SLA = external contract
- Error budget = $100\% - \text{SLO}$; fast-burn + slow-burn alerts

Glossary

Horizontal scaling

Adding more machines and distributing load, rather than upgrading one machine.

CAP theorem

A distributed store can guarantee at most two of Consistency, Availability, Partition tolerance — and partition tolerance is effectively mandatory.

Sharding

Splitting a dataset across multiple servers by key, for write and storage scaling.

Replication

Keeping full copies of data on multiple servers, for redundancy and read scaling.

Cache-aside

The application checks the cache, and on a miss, reads the DB and populates the cache itself.

Thundering herd / cache stampede

Many concurrent requests missing the same expired cache key at once, all hitting the DB simultaneously.

Idempotency key

A client-supplied unique identifier letting a server safely ignore a duplicate/retried request.

Dead-letter queue (DLQ)

Where messages that repeatedly fail processing are routed, instead of being retried forever.

SLI / SLO / SLA

Measured indicator / internal target / external contractual commitment, in increasing order of formality.

Error budget

The allowed amount of unreliability ($100\% - \text{SLO}$) before reliability work takes priority over feature work.

RTO / RPO

Recovery Time Objective (how long can we be down) / Recovery Point Objective (how much data can we lose).

Zero-trust architecture

Every service authenticates and authorizes every call it receives, including from other internal services.

Fanout-on-write vs. fanout-on-read

Precomputing a result on write (push) vs. computing it live on every read (pull).

Day-before-the-interview checklist

- [] Re-read the capacity estimation worksheet (Section 8) – practice one from scratch
- [] Re-derive the fanout-on-write vs. fanout-on-read trade-off without looking (Section 12)
- [] Re-derive the CAP "partition tolerance is mandatory" framing in your own words (Section 3.1)
- [] Practice narrating trade-offs OUT LOUD, not just thinking them – this is a rehearsed skill
- [] Review the four golden signals + SLI/SLO/SLA distinction (Section 6)
- [] Pick one case study and do a full 45-minute timed run-through, framework step by step
- [] Prepare 2-3 clarifying questions you ask by default, regardless of the prompt
- [] Sleep. A tired brain cannot narrate trade-offs coherently, no matter how well prepared.

Sources consulted

Compiled from current practitioner and authoritative sources as of September 2026. Where this guide says a question or problem is "commonly asked" or "frequently reported," that reflects language or data found directly in the cited source, not an invented popularity claim.

GLASSDOOR & INTERVIEW-EXPERIENCE FORUMS

[Glassdoor — Grafana Labs Senior Backend Engineer Interview](#)

[Glassdoor — Systems Design Interview Questions \(aggregated\)](#)

[Blind — "What is the point of back-of-the-envelope QPS calculations?"](#)

GITHUB REPOSITORIES & HANDBOOKS

[donnemartin/system-design-primer \(85,000+ stars\)](#)

GEEKSFORGEEKS & STRUCTURED PREP GUIDES

[GeeksforGeeks — Top Most Asked System Design Interview Questions](#)

[GeeksforGeeks — Most Commonly Asked System Design Interview Problems](#)

[GeeksforGeeks — Load Balancer System Design Interview Question](#)

[GeeksforGeeks — Designing WhatsApp Messenger](#)

[Data Interview — Top 23 Distributed Systems Interview Questions](#)

CLOUD ARCHITECTURE & OFFICIAL DOCS

[AWS Well-Architected Framework — Reliability Pillar](#)

[AWS — The 6 Pillars of the Well-Architected Framework](#)

PRACTITIONER & ENGINEERING BLOGS

[Hello Interview — System Design in a Hurry](#)

[Hello Interview — CAP Theorem for System Design Interviews](#)

[Hello Interview — Kafka Deep Dive](#)

[Hello Interview — API Gateway Deep Dive](#)

[Hello Interview — Design a Messaging App Like WhatsApp](#)

[Design Gurus — The System Design Interview Rubric Nobody Shows You](#)

[Design Gurus — System Design Interview Expectations by Company and Level](#)

[Splunk — SRE Metrics: The Four Golden Signals](#)

[techinterview.org — Twitter/X News Feed: Fanout, Celebrity Problem, Ranking](#)

[Formation — How to Pass System Design Interviews: Drive the Design](#)

[DEV Community — The Hidden Reason Most Candidates Fail System Design Interviews](#)

[Design Gurus — How Top Candidates Clarify Requirements Before Designing](#)

Systems at Scale Manual · HLD / System Design Edition · Use the "Save as PDF" button (top of page) to export via your browser's print dialog for offline study.